



TRAILHEAD
TECHNOLOGY PARTNERS

Oops, I Leaked It Again

API Security Mistakes Fixed



Jonathan "J." Tower

{
"celebrities":
 "Tom Hanks",
 "Beyoncé",
 "Leonardo DiCaprio",
 "Taylor Swift",
 "Dwayne Johnson",
 "Scarlett Johansson"
}



8 common API
security mistakes,
and **practical fixes** to keep
your REST APIs safe

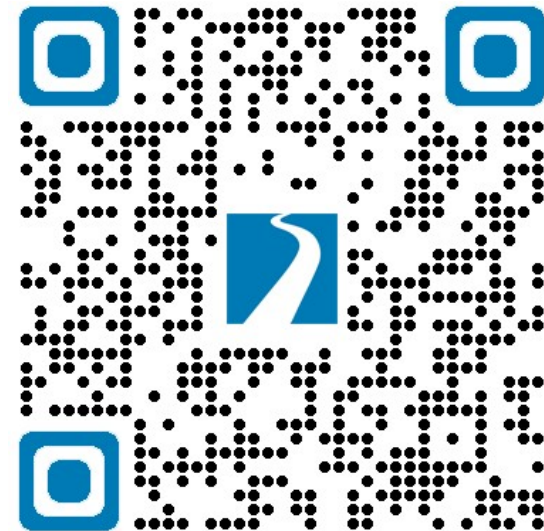
Jonathan "J." Tower

Partner & Principal Consultant



- ❑ Microsoft MVP in .NET
- ❑ jtower@trailheadtechnology.com
- ❑ trailheadtechnology.com/blog
- ❑ [jtowermi](#)
- ❑ Jonathan "J." Tower

EXPERT Consultation

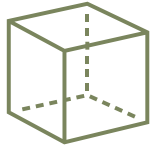


bit.ly/th-offer

github.com/trailheadtechnology/api-security

8 Most Common API Security Mistakes

8 Common Issues



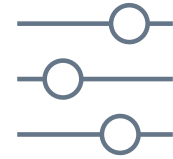
Broken Object
Level Authz



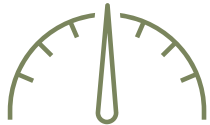
Broken Object
Prop Level Authz



Broken
Authentication



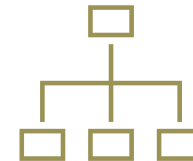
Misconfig &
Insecure Defaults



Unrestricted Resource
Consumption



Verbose Errors or
Logging Leaks



Unrestricted Resource
Consumption



Transport Layer &
Secrets Mgmt

Examples



.NET

Examples



Broken Object Level Authorization

Common Mistake #1

Broken Object Level Authorization



```
//[Authorize]  
[HttpGet("{id:guid}")]  
public Task<User?> GetById(Guid id) => _db.Users.FindAsync(id).AsTask();
```

Broken Object Level Authorization



```
[ApiController]  
[Authorize]  
[Route("api/[controller]")]  
public abstract class BaseApiController : ControllerBase { ... }
```

Broken Object Level Authorization



```
public sealed class OnlyBaseApiControllerFeatureProvider
    : IApplicationFeatureProvider<ControllerFeature>
{
    public void PopulateFeature(IEnumerable<ApplicationPart> parts, ControllerFeature feature)
    {
        var allowed = feature.Controllers
            .Where(t => typeof(BaseApiController).IsAssignableFrom(t.AsType()))
            .ToList();

        var rejected = feature.Controllers.Except(allowed).ToList();
        feature.Controllers.Clear();
        foreach (var c in allowed) feature.Controllers.Add(c);

        if (rejected.Count > 0)
            throw new InvalidOperationException(
                $"Controllers must inherit {nameof(BaseApiController)}. Offenders: " +
                string.Join(", ", rejected.Select(r => r.FullName)));
    }
}
```

Broken Object Level Authorization



```
builder.Services.AddControllers()  
    .ConfigureApplicationPartManager(pm =>  
        pm.FeatureProviders.Add(new OnlyBaseApiControllersFeatureProvider()));
```


Broken Object Level Authorization



```
// Client posts userId/tenantId; server uses it directly
public async Task<IResult> Update(Guid id, UpdateOrderDto dto)
{
    var order = await db.Orders.SingleOrDefaultAsync(o => o.Id == id
        && o.OwnerUserId == dto.UserId); // ✗ client reports user
    // ...
}
```

Broken Object Level Authorization



```
public async Task<IResult> Update(Guid id, UpdateOrderDto dto,
    ClaimsPrincipal user)
{
    var userId = user.FindFirst("sub")?.Value!;

    var order = await db.Orders.SingleOrDefaultAsync(o => o.Id == id
        && o.OwnerUserId == userId);
    // ...
}
```

Broken Object Property Level Authorization

Common Mistake #2

Broken Object Property Level Authorization

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Secrets
and
Credentials

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Privilege Hints & Implementation Details

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Heavy or
Unused Fields

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Expose These
Thoughtfully

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```


Mass Assignment

Mass Assignment

```
public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    public bool IsAdmin { get; set; }

    public Guid OwnerId { get; set; }
}
```

Mass Assignment



Mass Assignment

```
public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    [System.Text.Json.Serialization.JsonIgnore]
    public bool IsAdmin { get; set; }

    [Microsoft.AspNetCore.Mvc.ModelBinding.BindNever]
    public Guid OwnerId { get; set; }
}
```

Mass Assignment



Mass Assignment

```
public class UpdateUserRequestDto
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    public bool IsAdmin { get; set; }

    public Guid OwnerId { get; set; }
}
```

Treat APIs Like UIs

```
{  
  "id": 123,  
  "Name": "Person Name"  
  "isDifficultUser": true  
}
```

ID:

Name:

Difficult:



Treat APIs Like UIs

```
{  
  "id": 123,  
  "Name": "Person Name"  
  "isVerified": true  
}
```

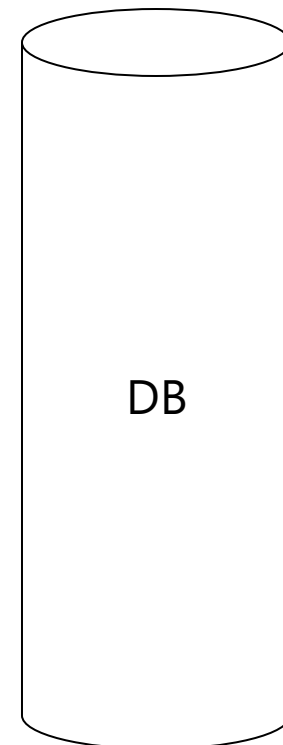
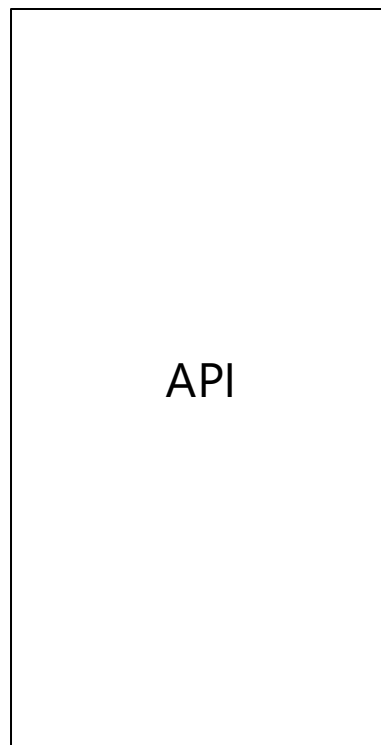
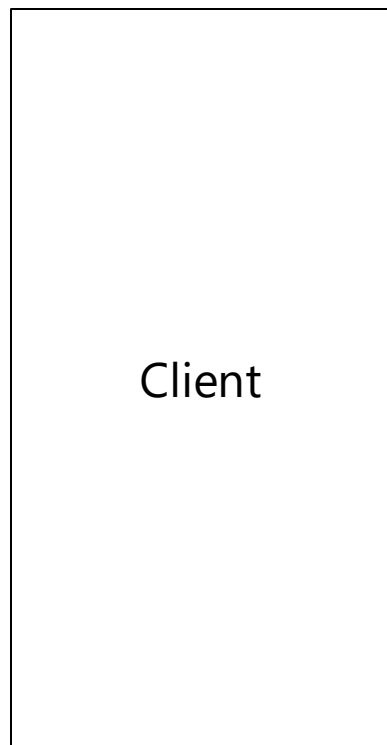
ID:

Name:

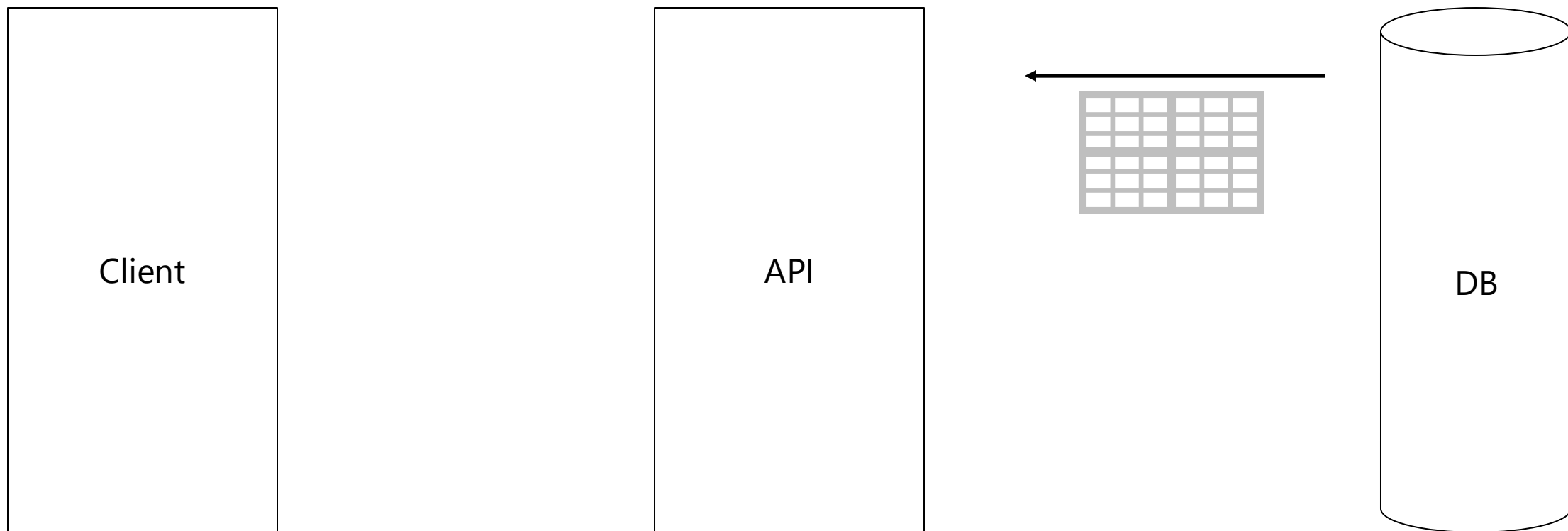
Verified:



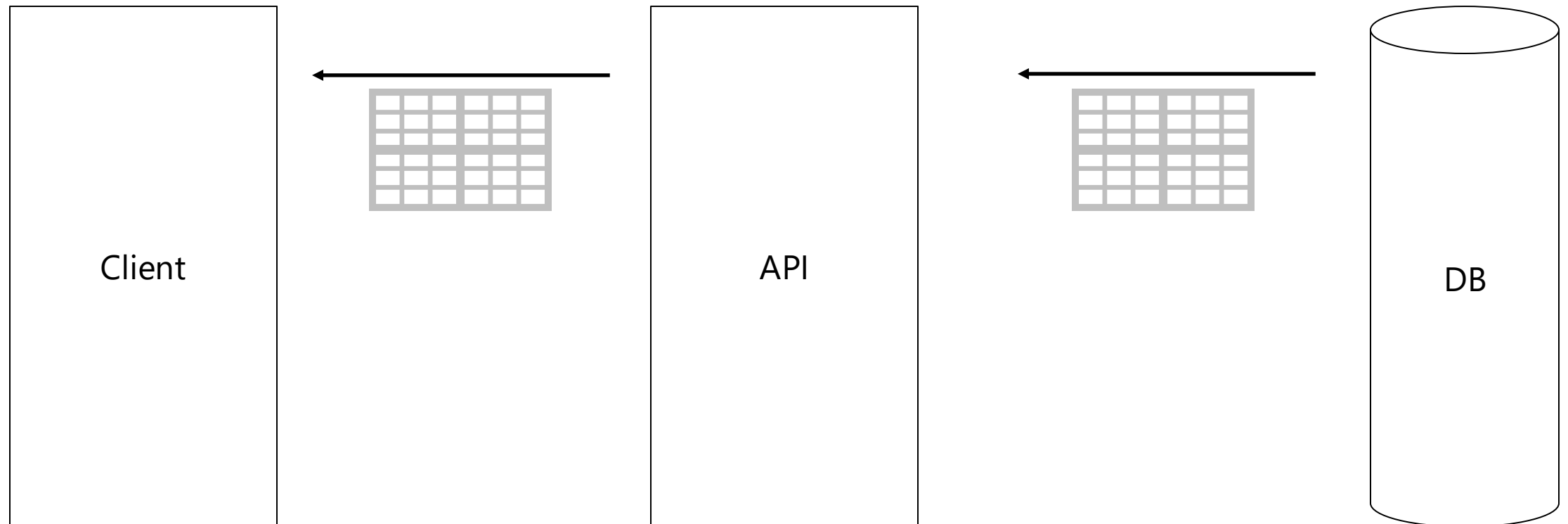
Use DTOs / ViewModels



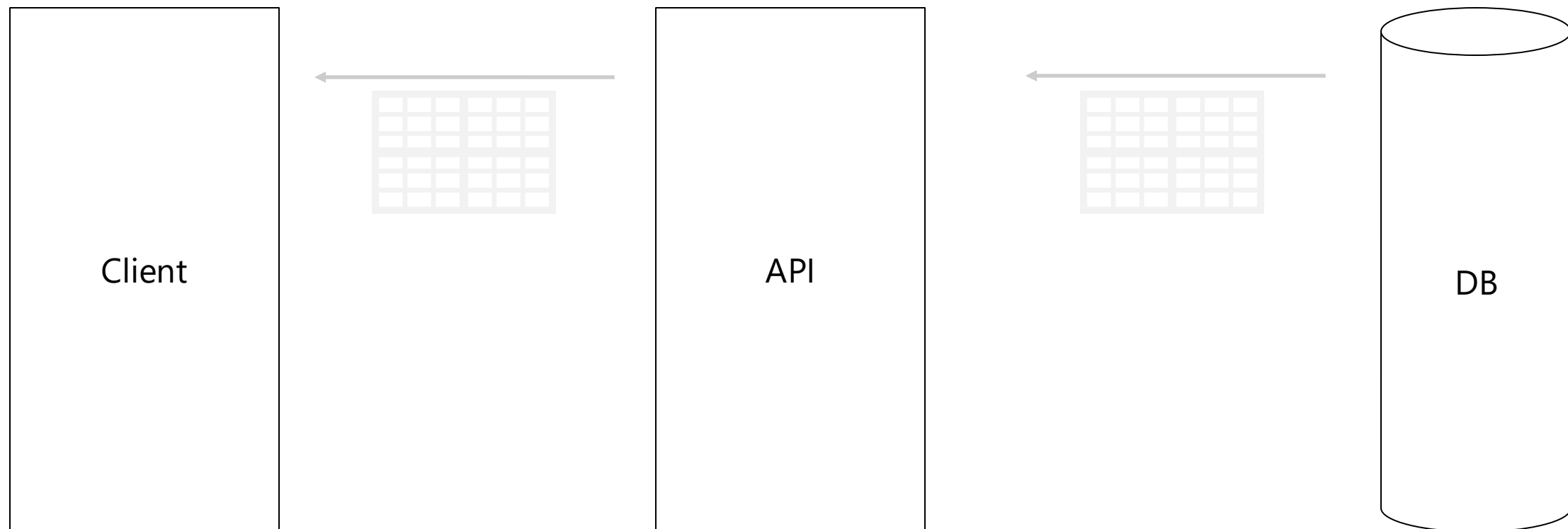
Use DTOs / ViewModels



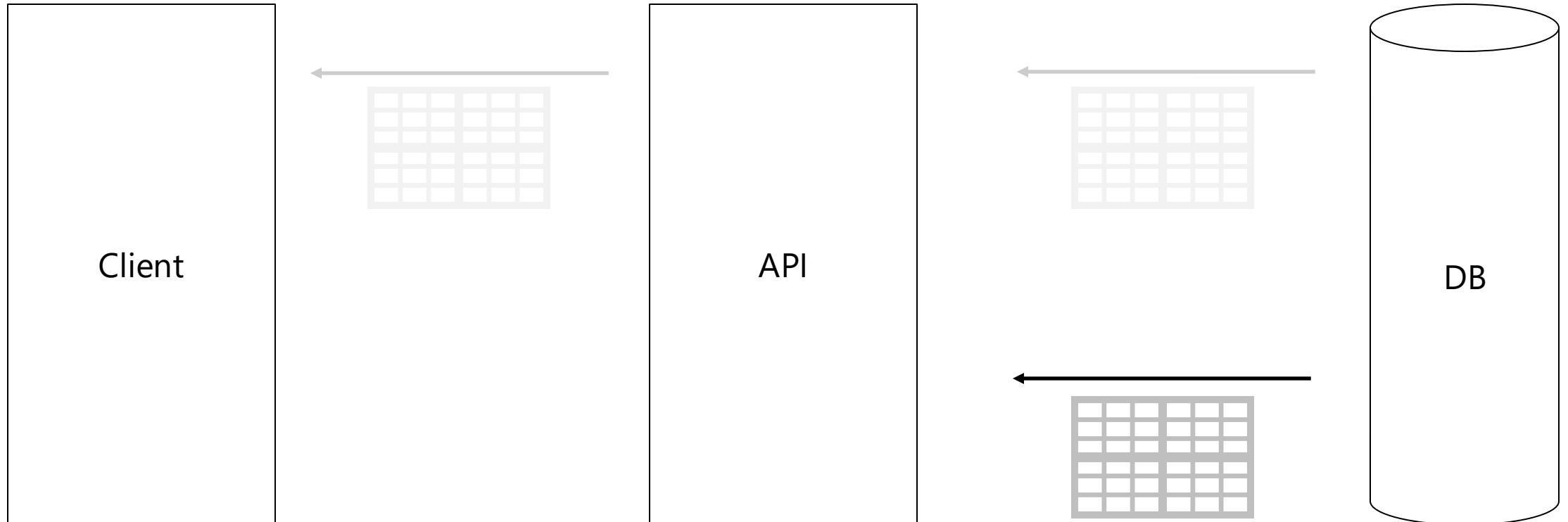
Use DTOs / ViewModels



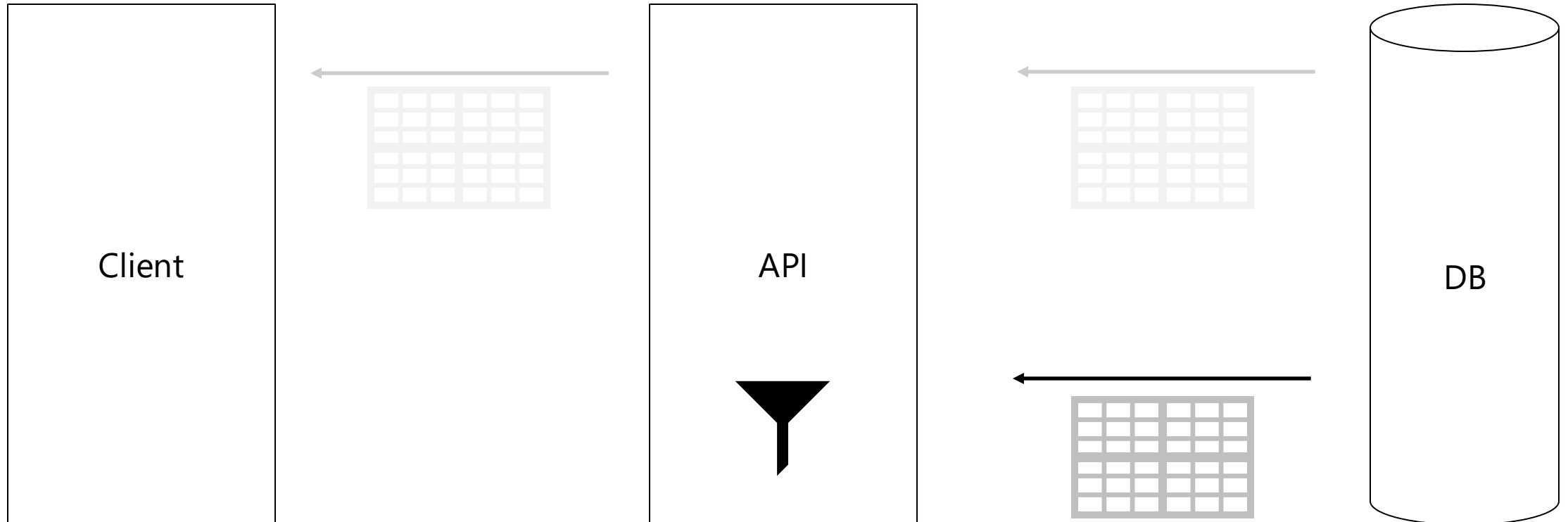
Use DTOs / ViewModels



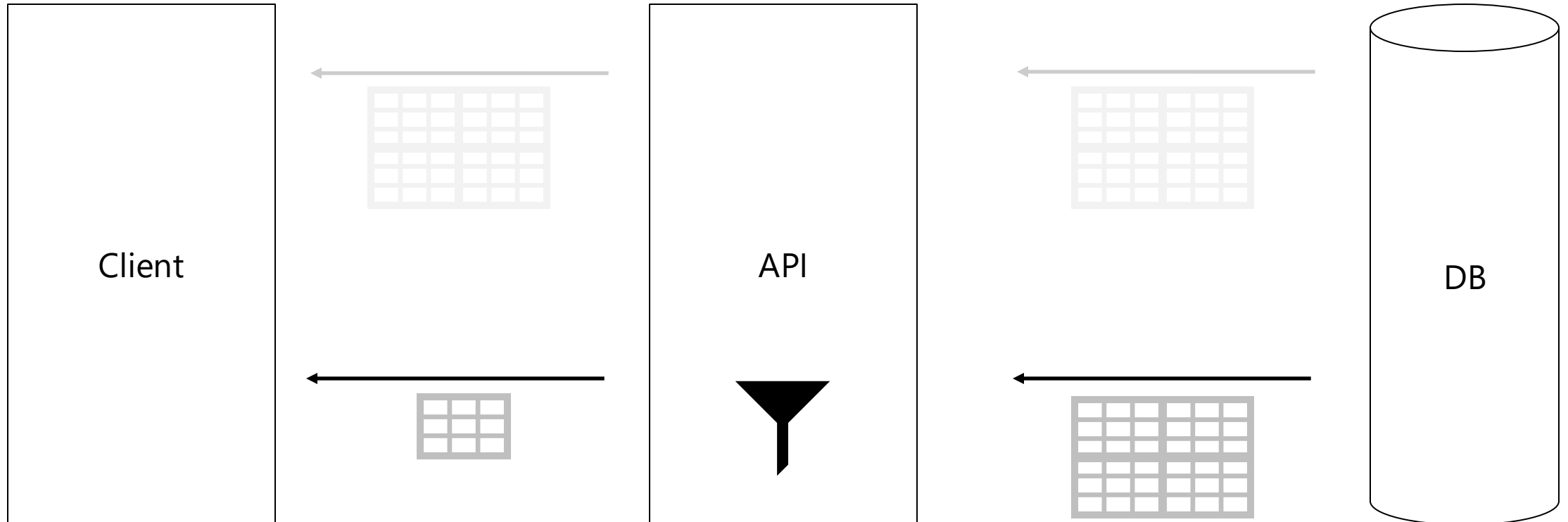
Use DTOs / ViewModels



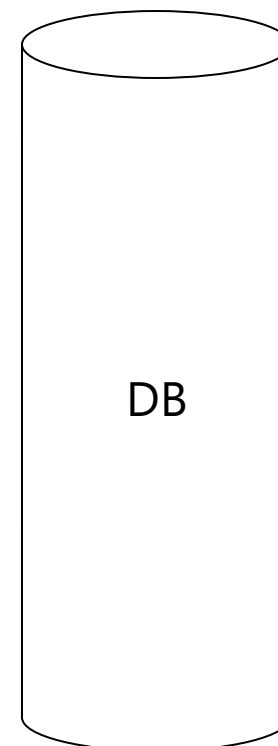
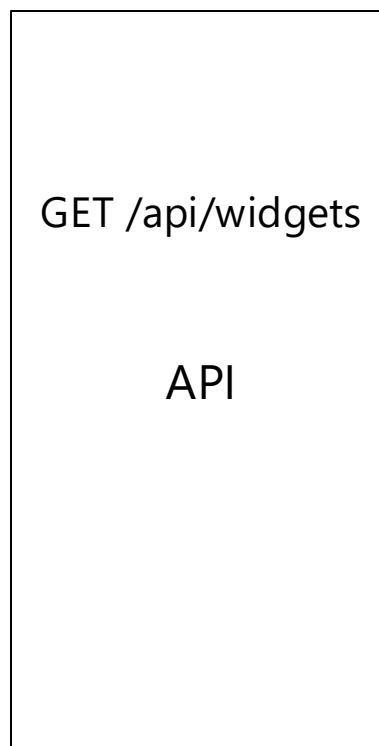
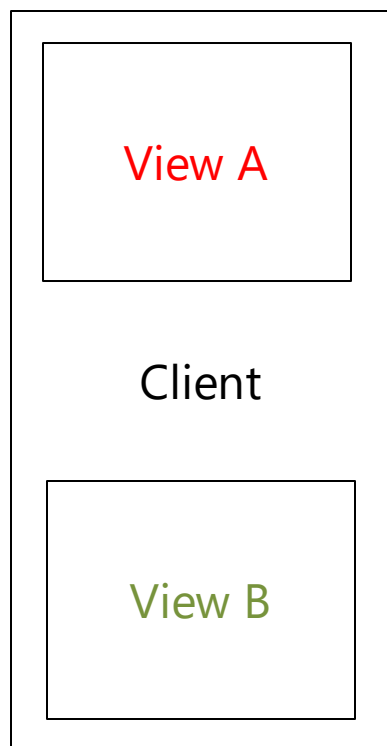
Use DTOs / ViewModels



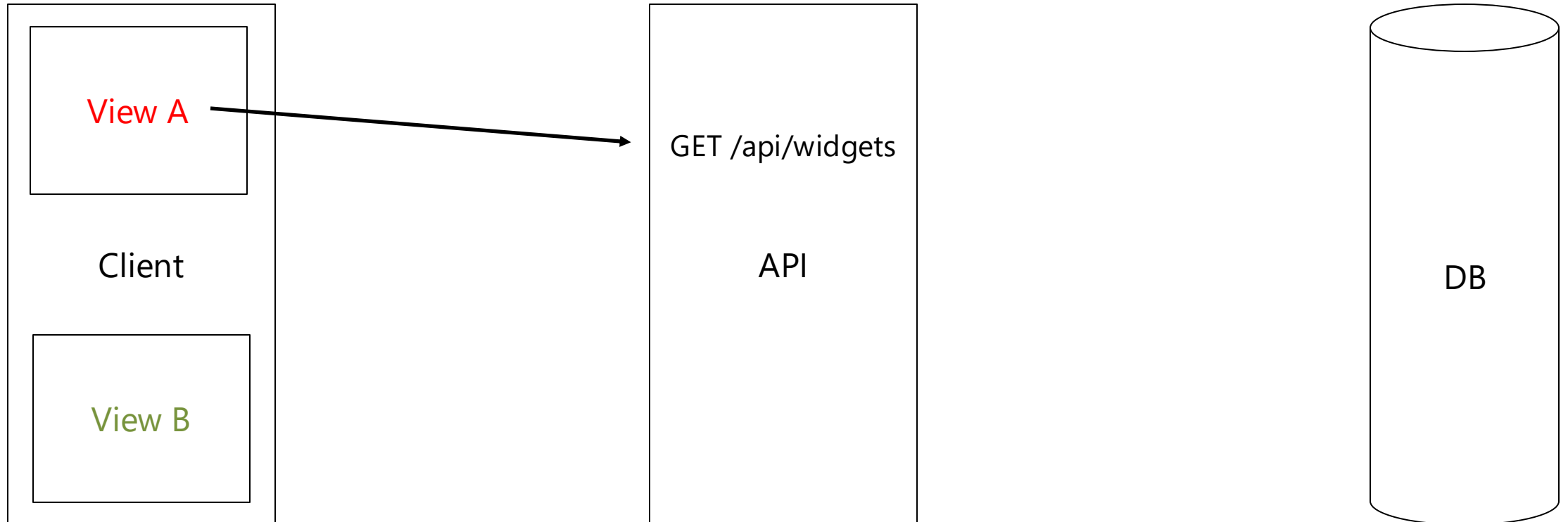
Use DTOs / ViewModels



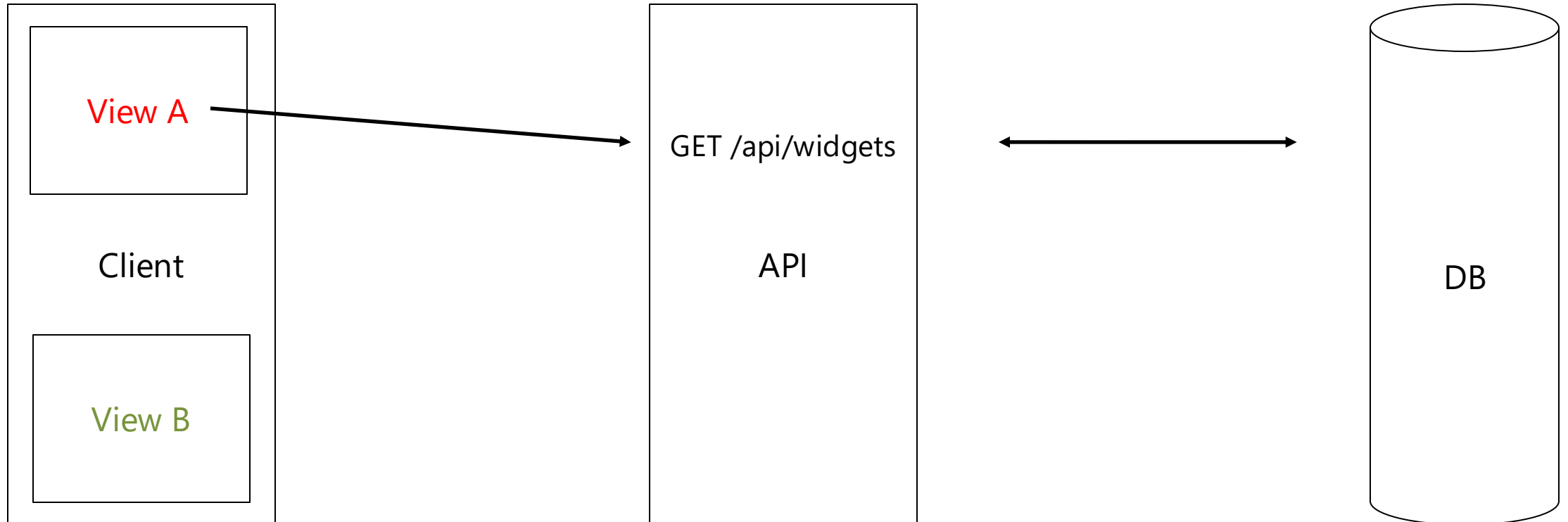
Avoid General APIs



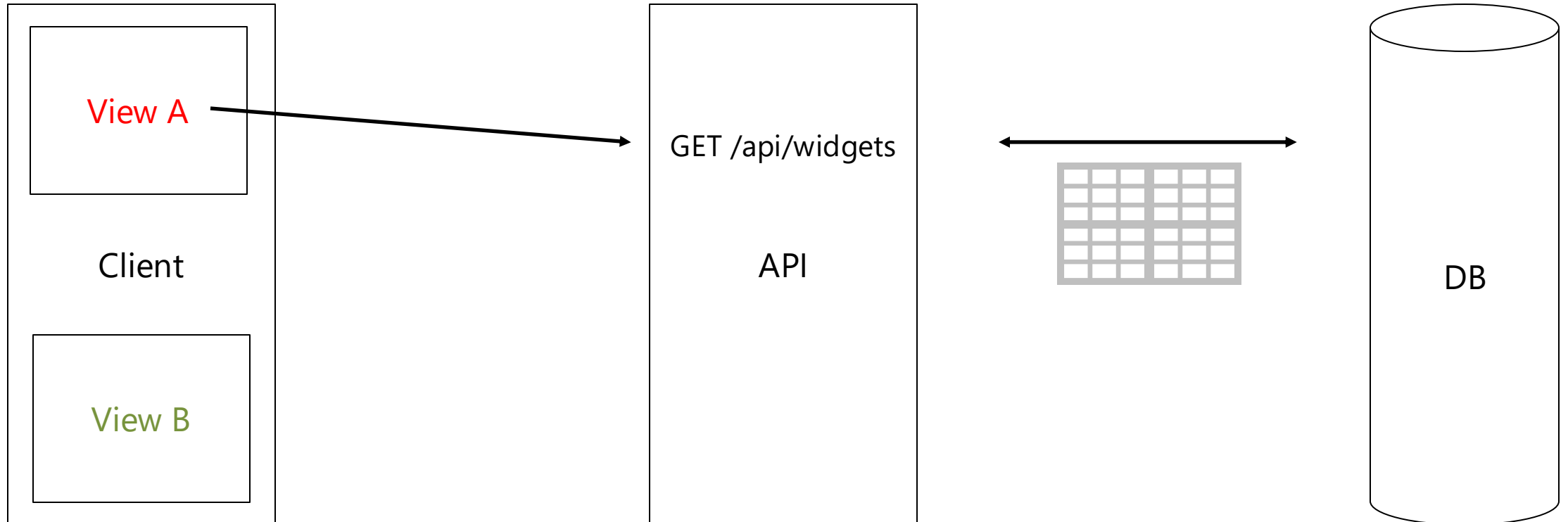
Avoid General APIs



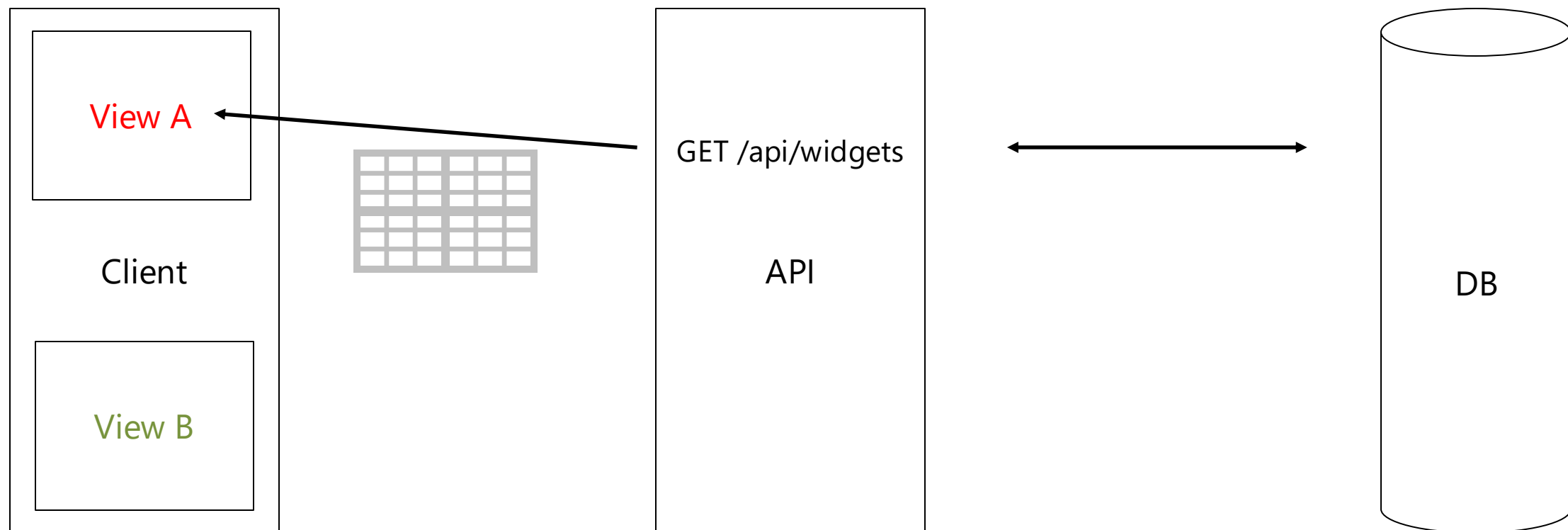
Avoid General APIs



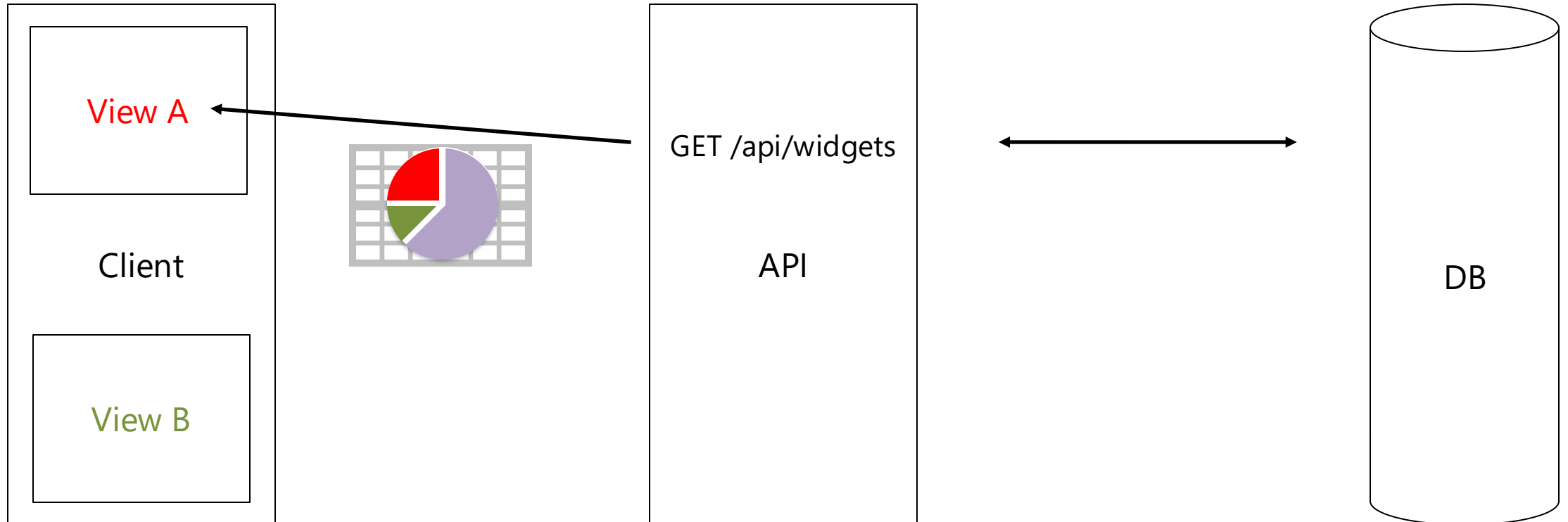
Avoid General APIs



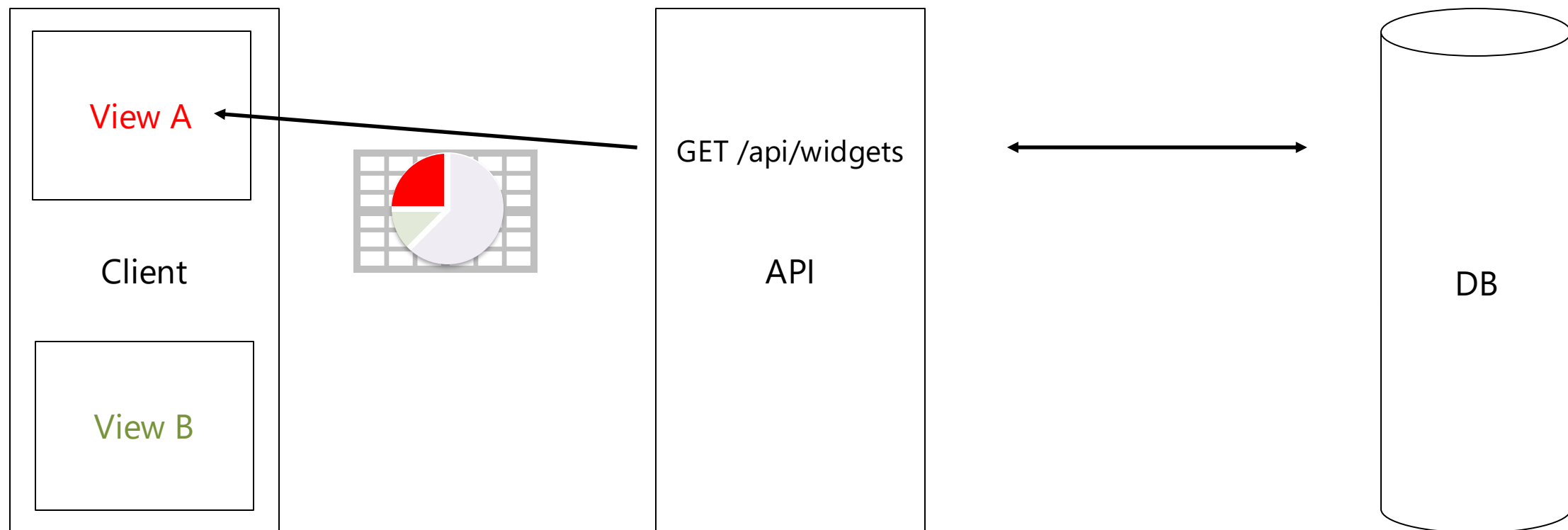
Avoid General APIs



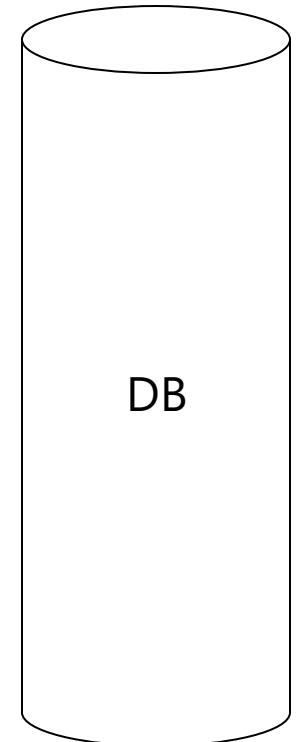
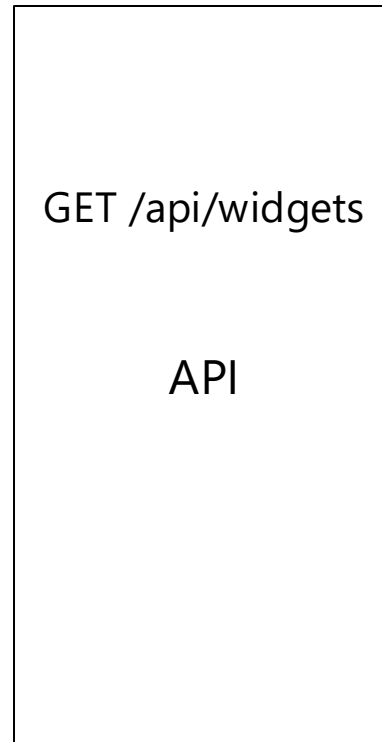
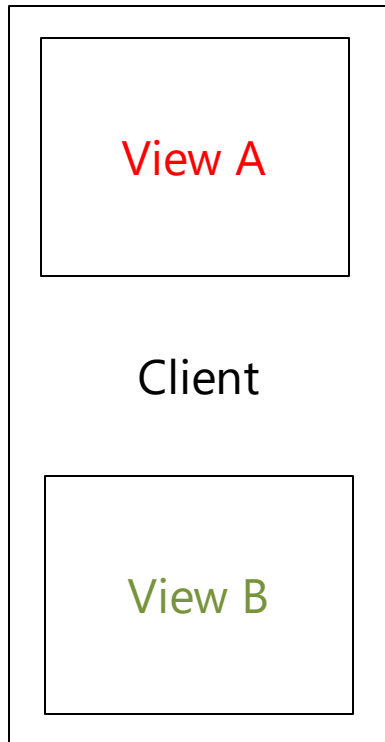
Avoid General APIs



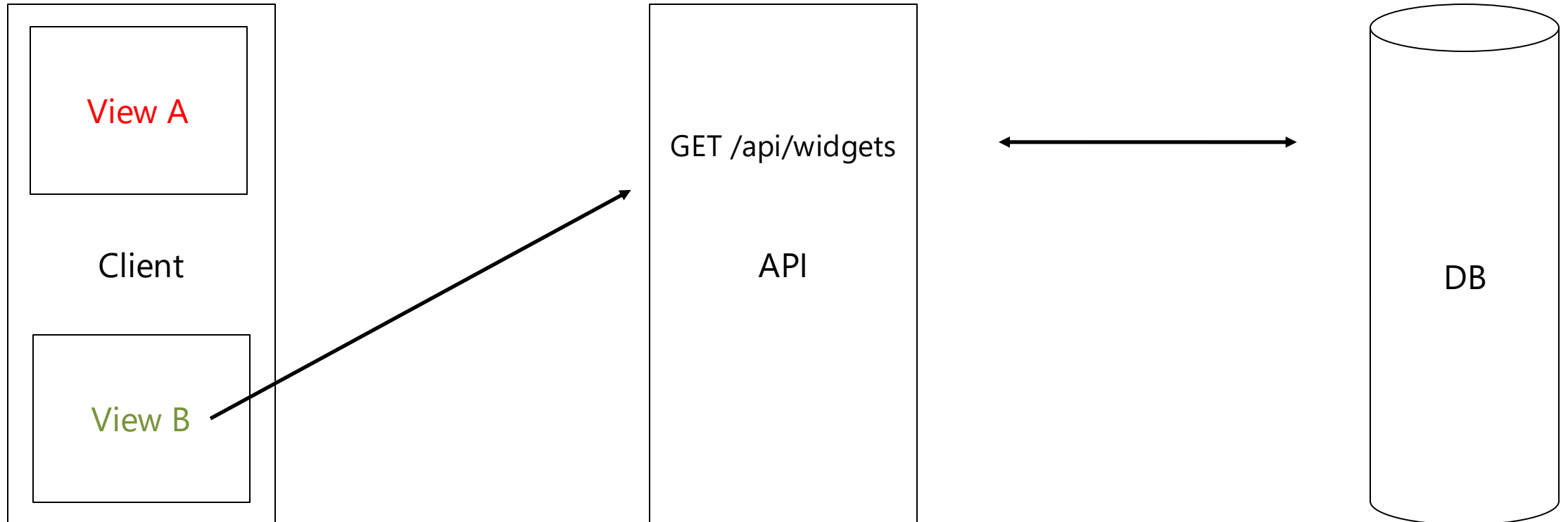
Avoid General APIs



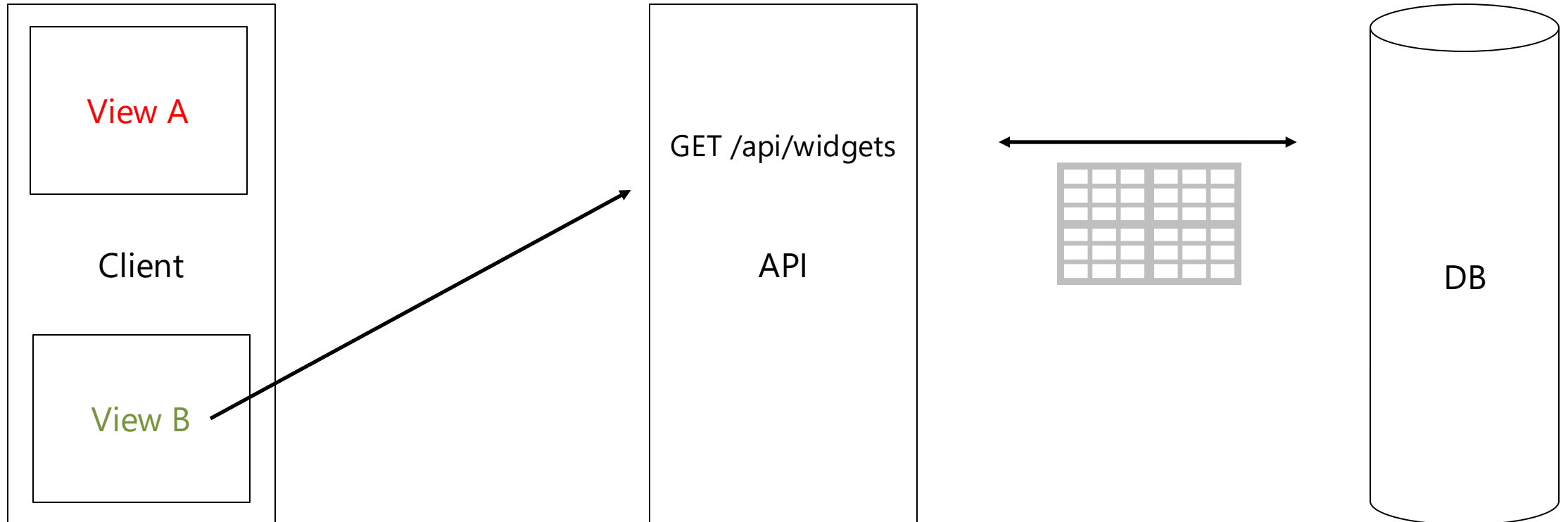
Avoid General APIs



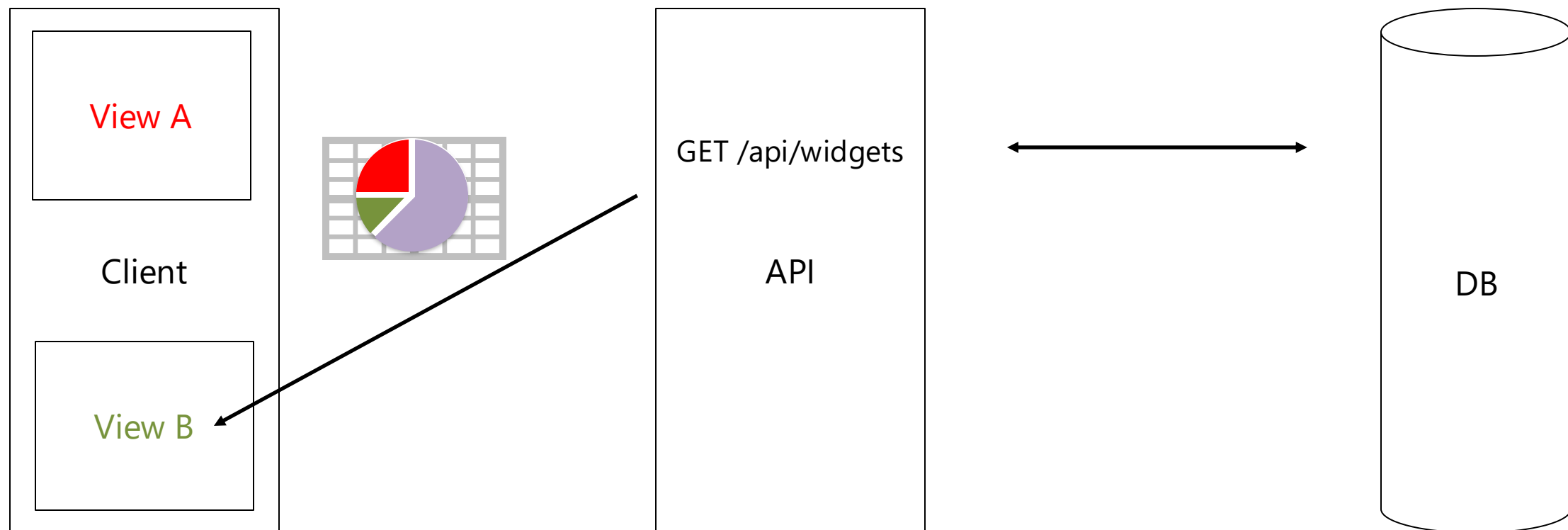
Avoid General APIs



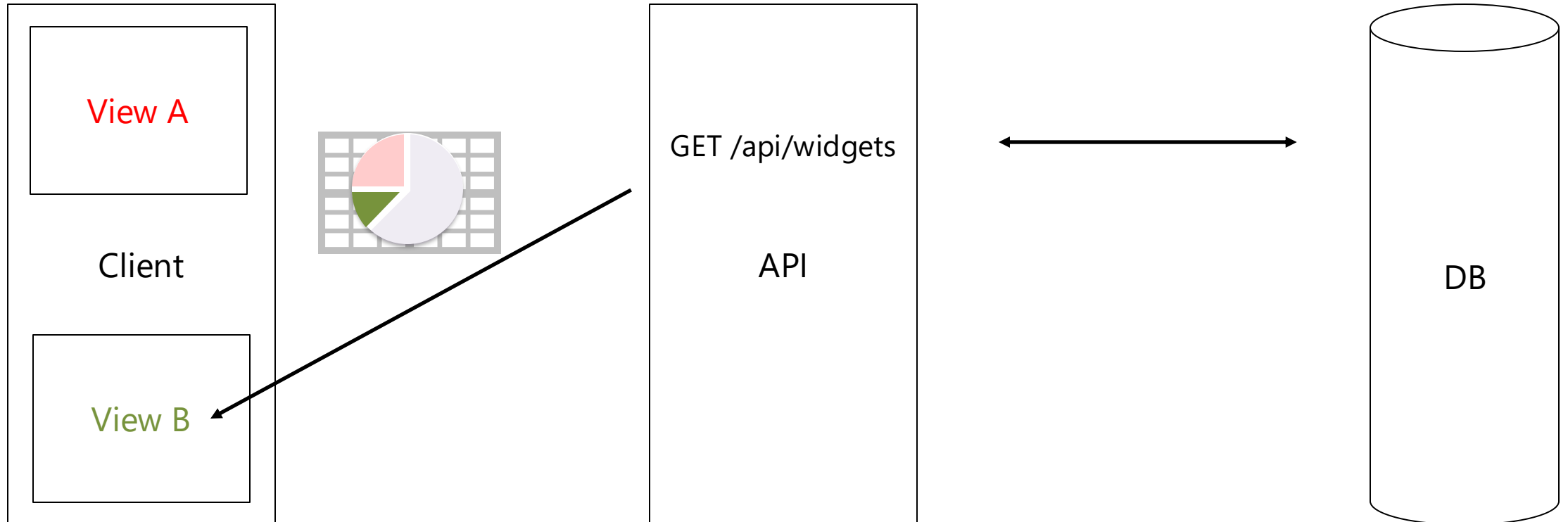
Avoid General APIs



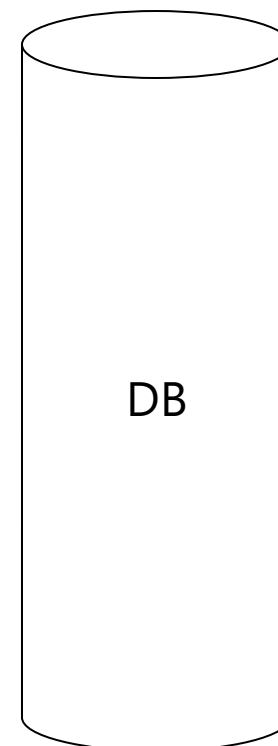
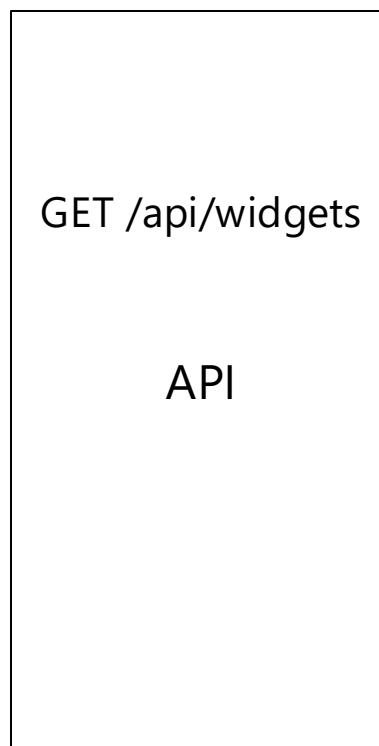
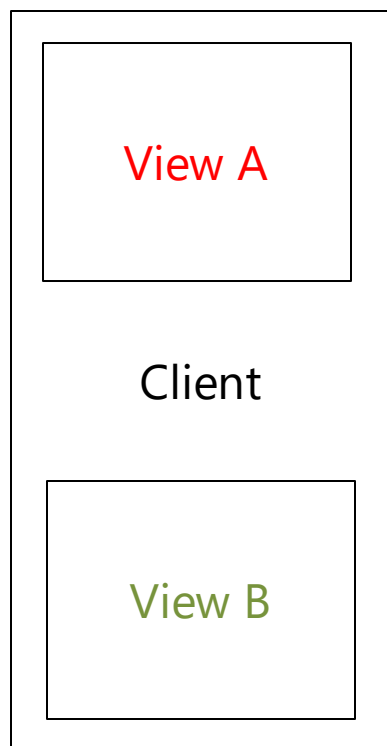
Avoid General APIs



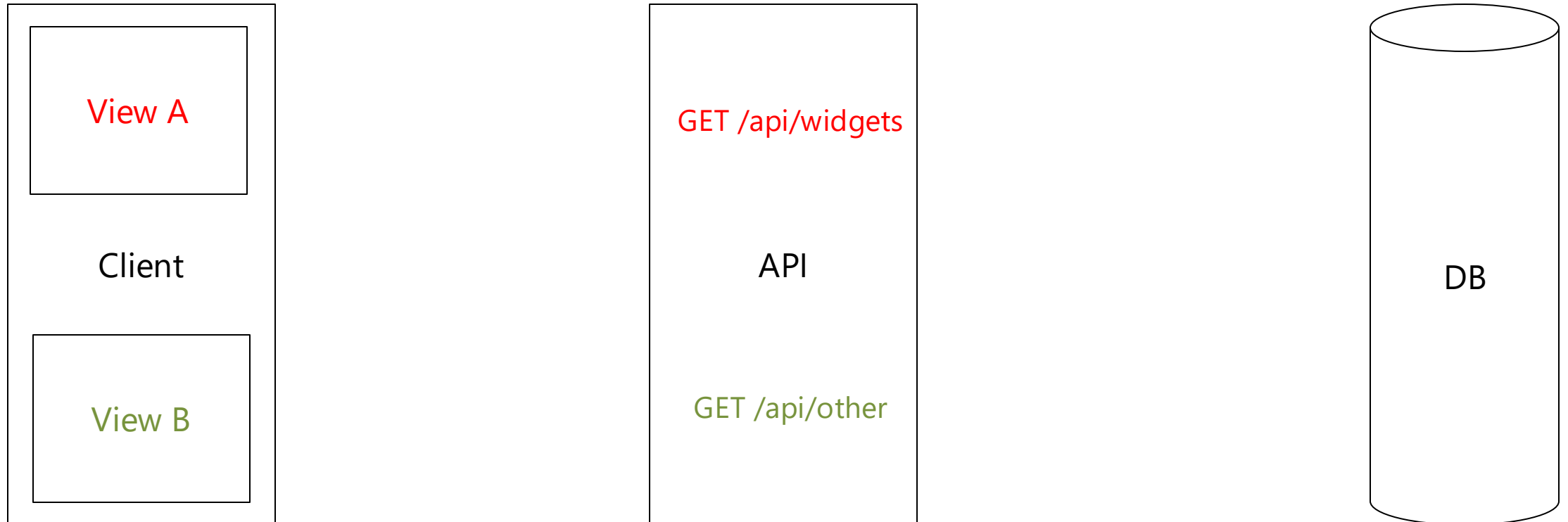
Avoid General APIs



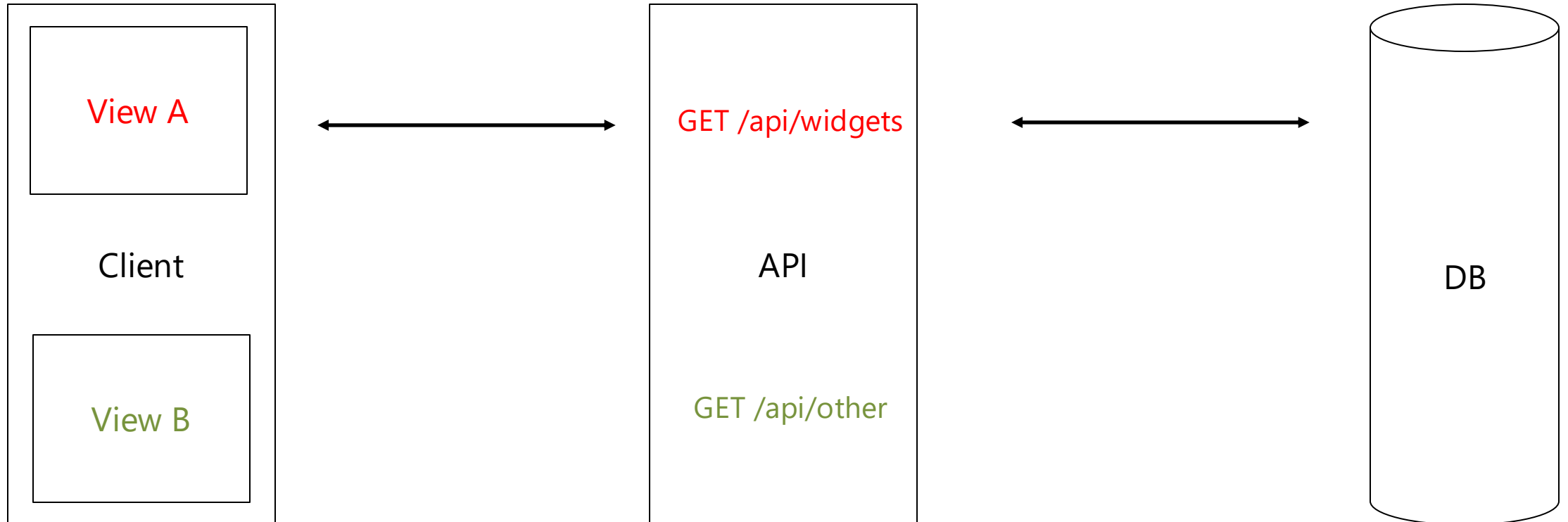
Avoid General APIs



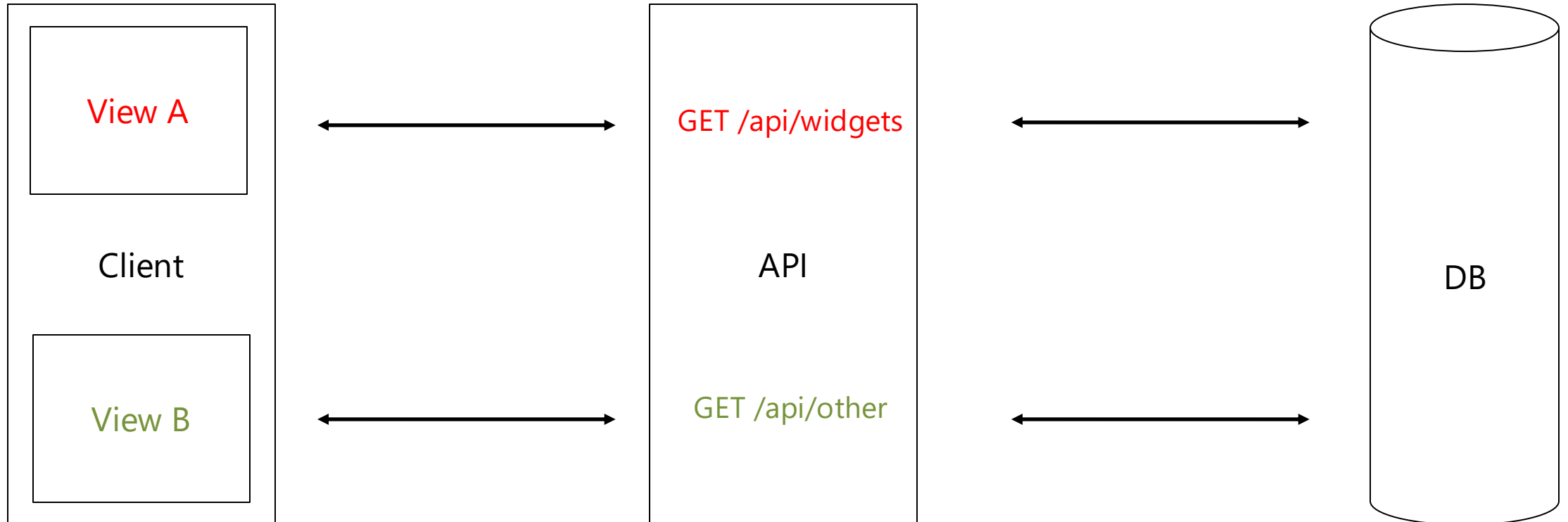
Avoid General APIs



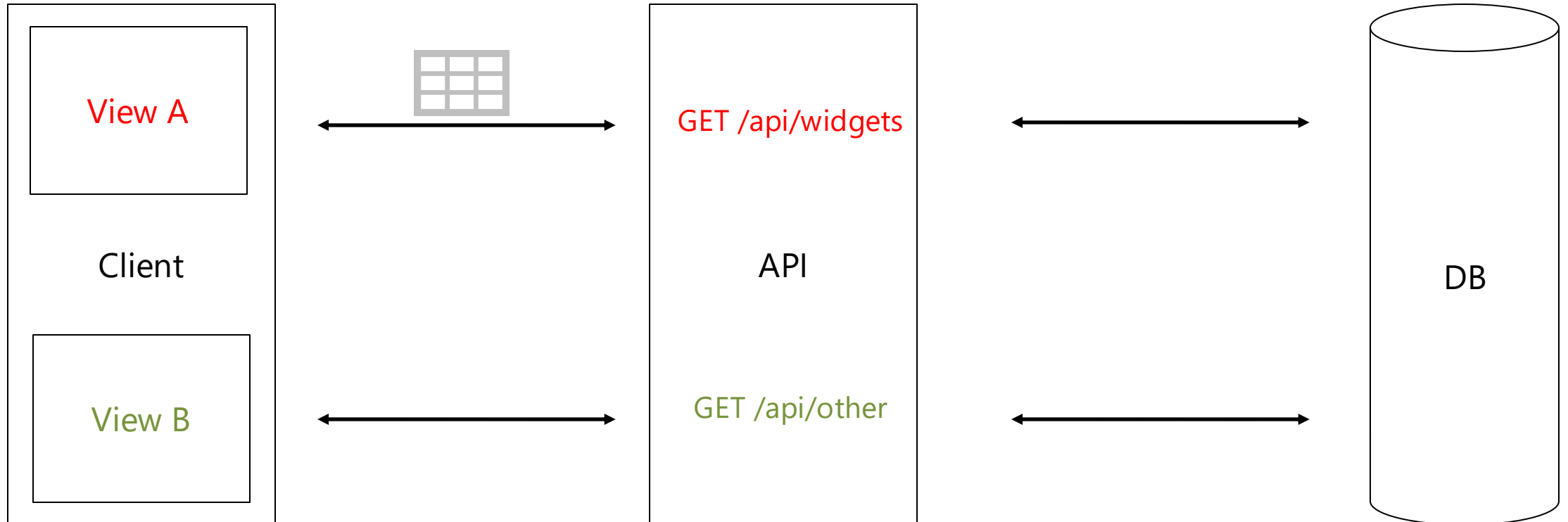
Avoid General APIs



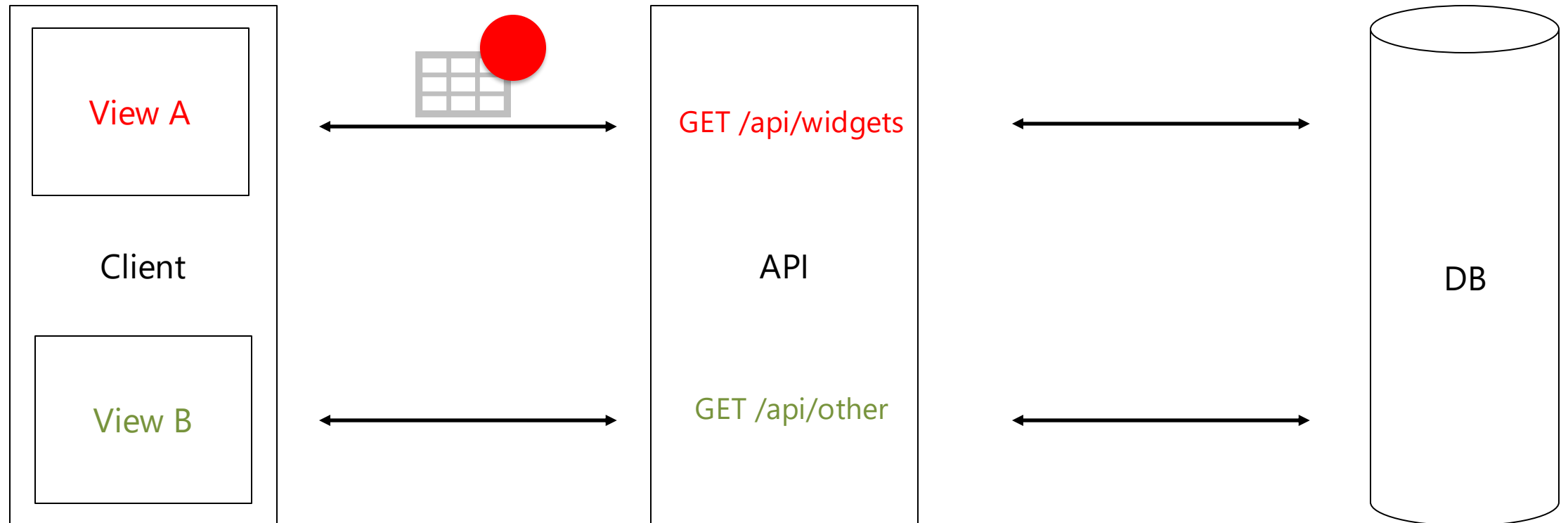
Avoid General APIs



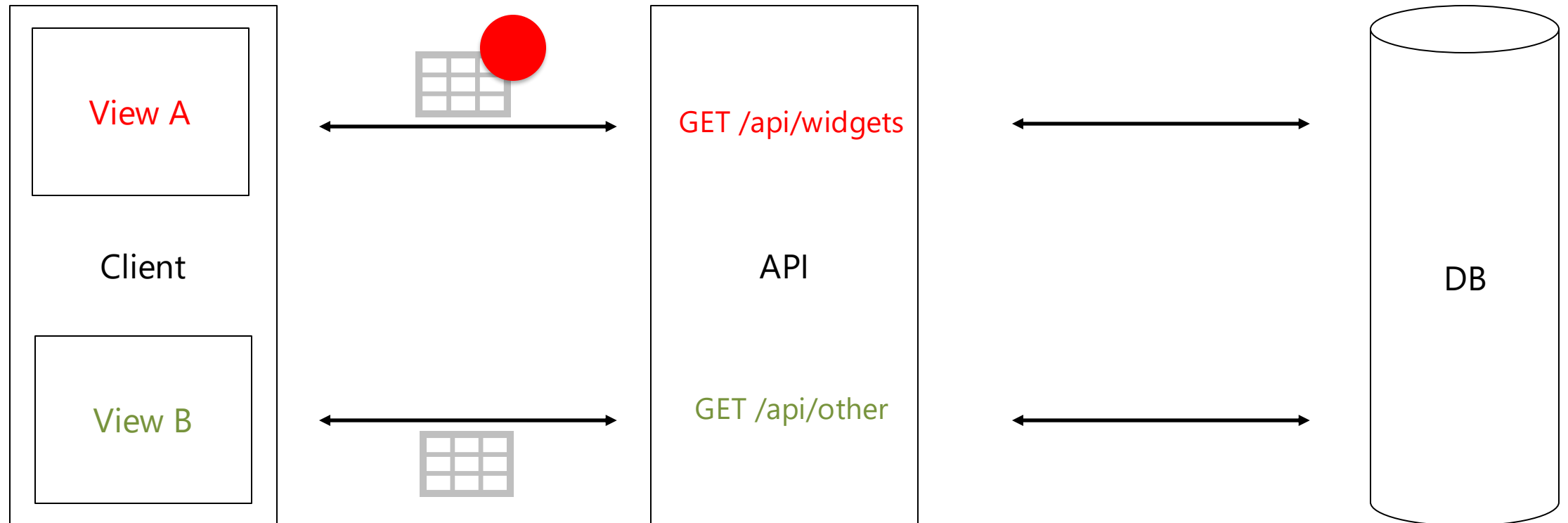
Avoid General APIs



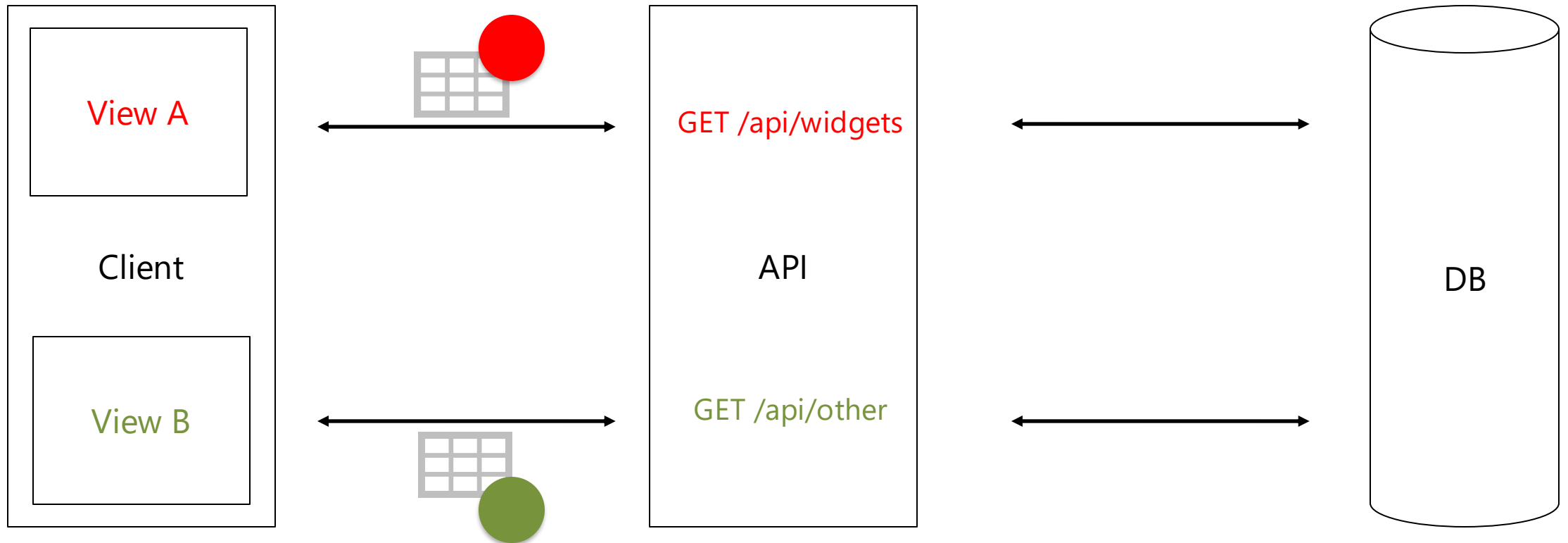
Avoid General APIs



Avoid General APIs



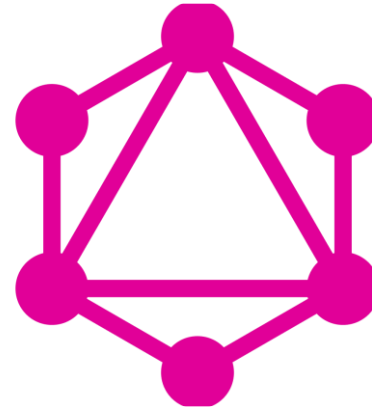
Avoid General APIs



Avoid General APIs ❌

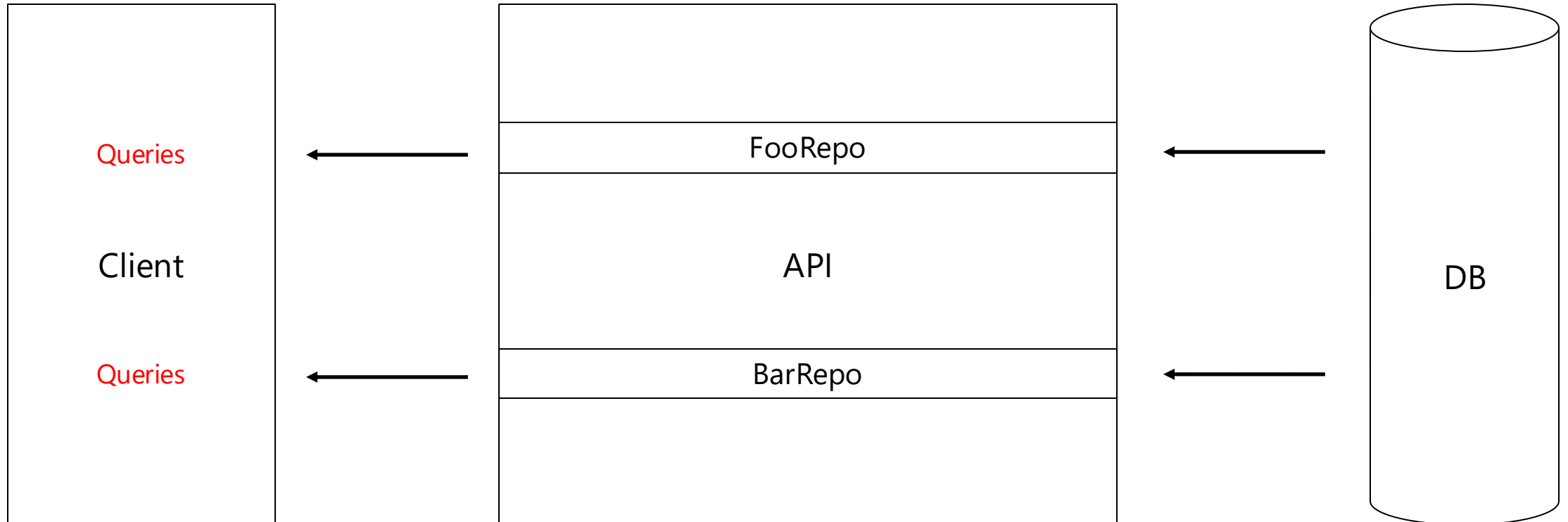


OData

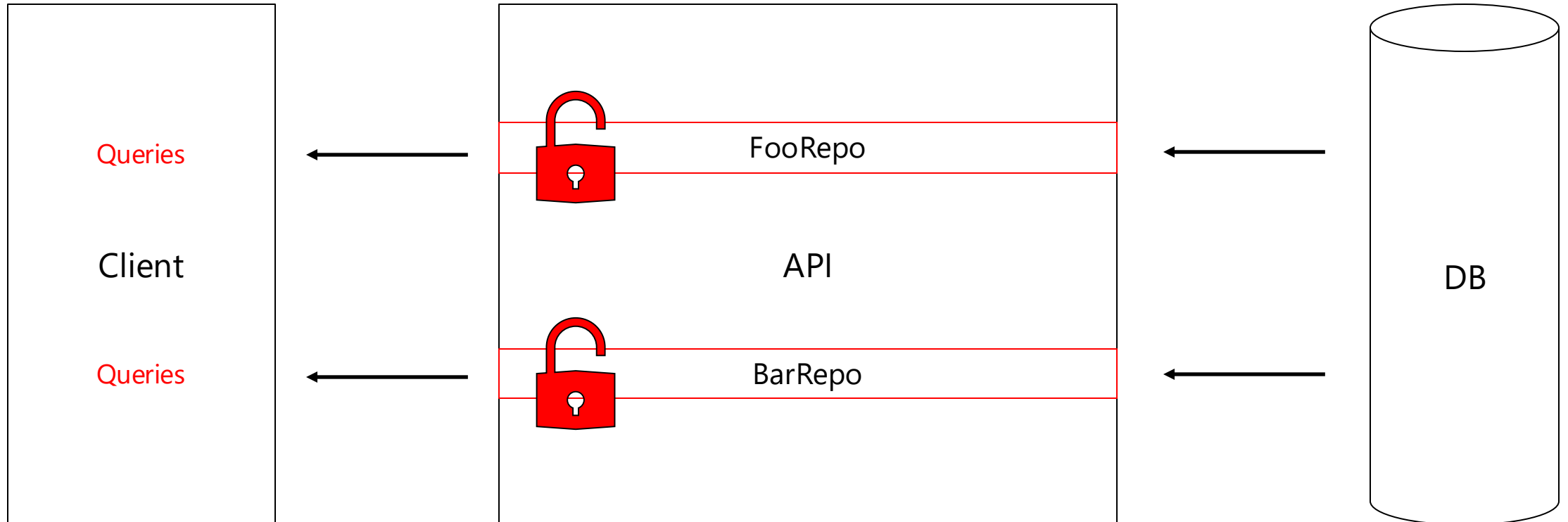


GraphQL

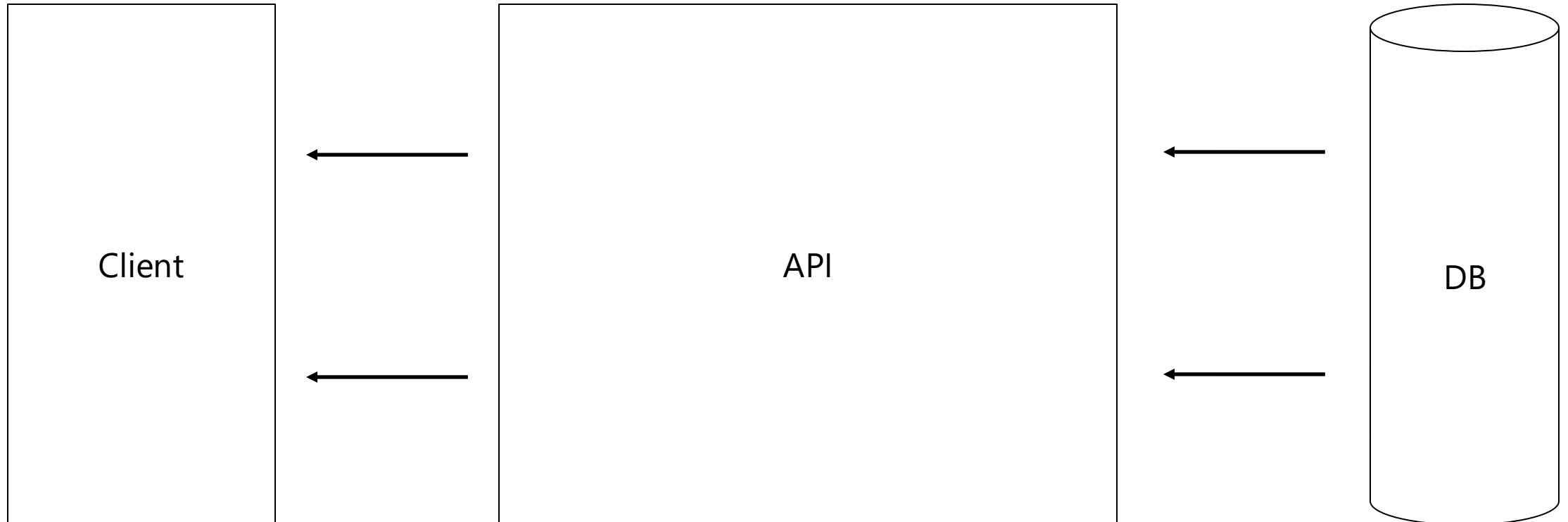
Avoid General APIs



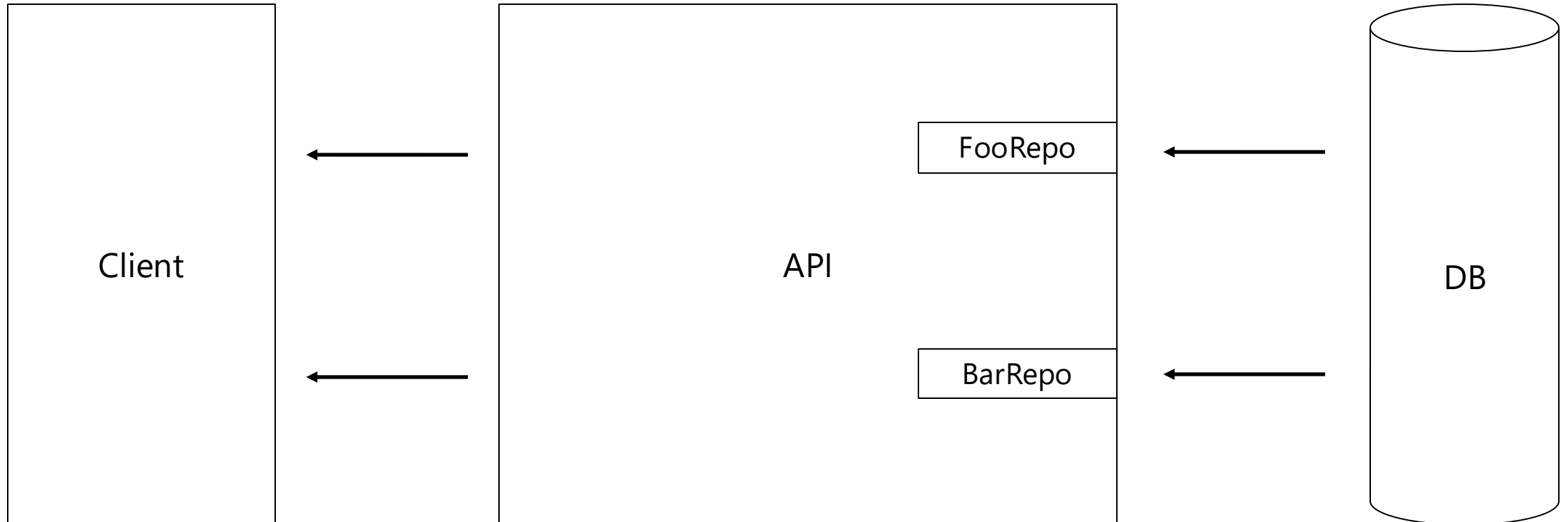
Avoid General APIs



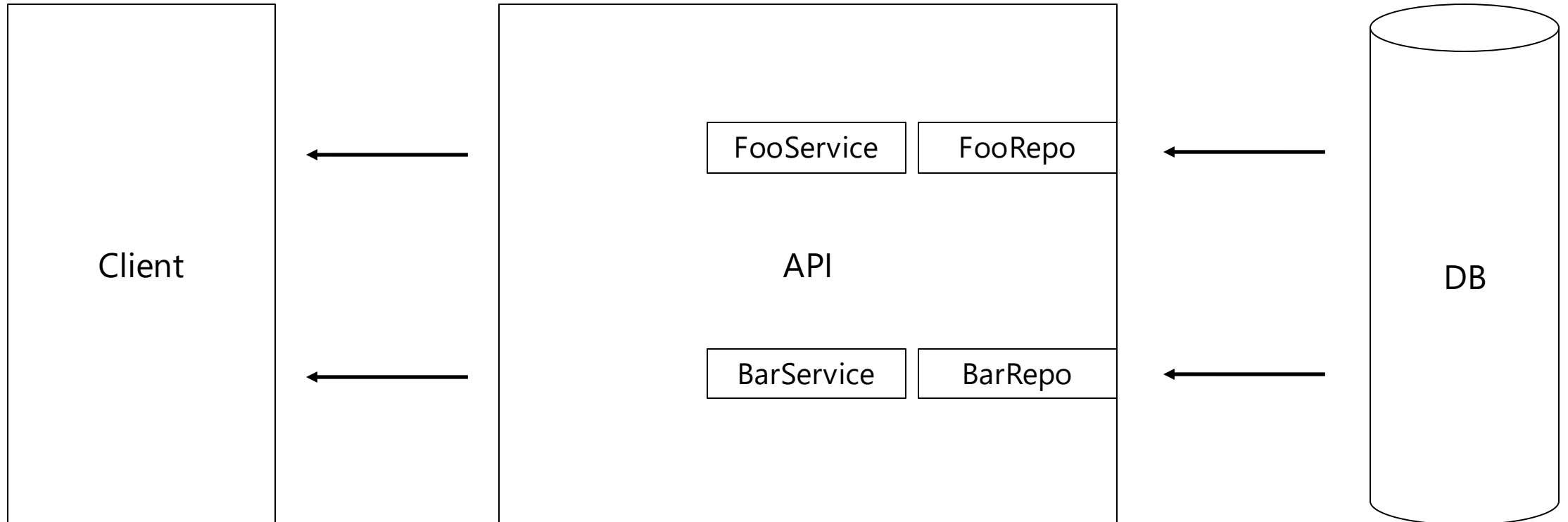
Avoid General APIs



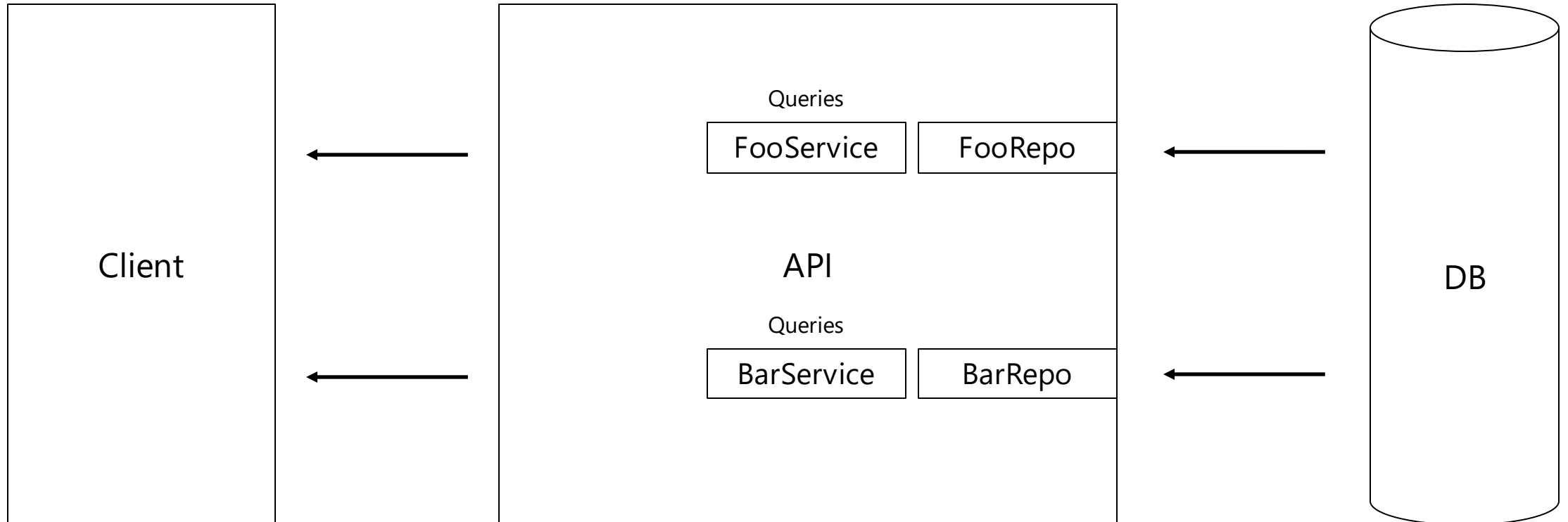
Avoid General APIs



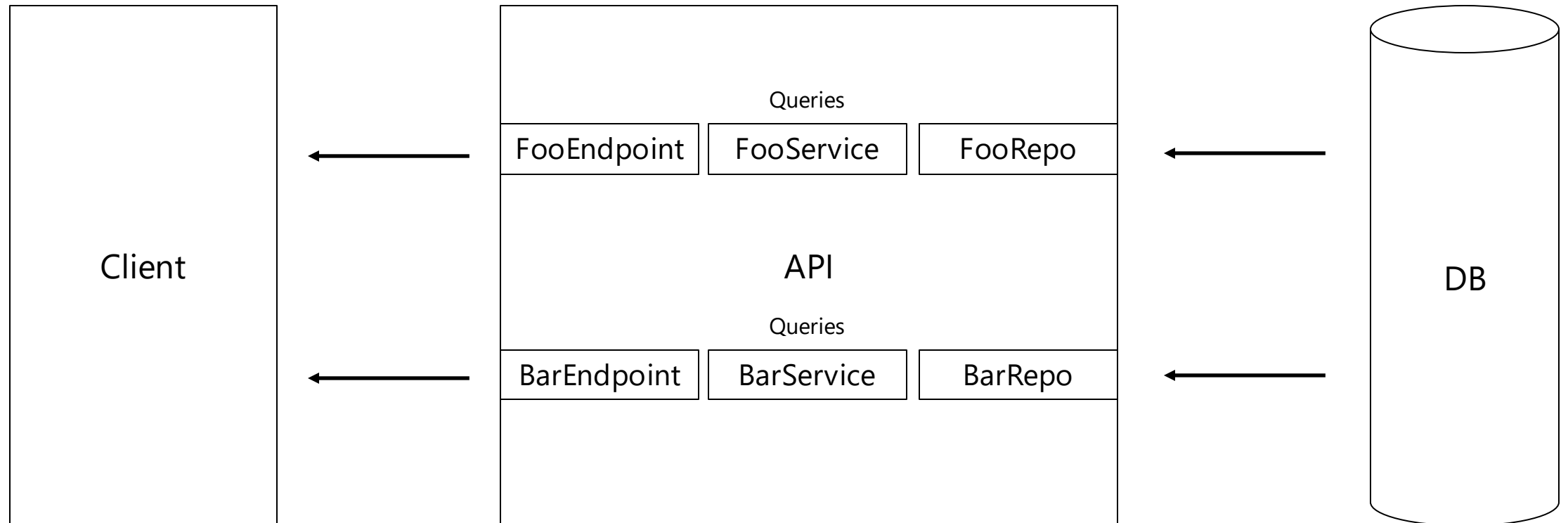
Avoid General APIs



Avoid General APIs



Avoid General APIs



Large Data Handling



```
{  
  ...  
  "profilePicture": "iVBORw0KGgoUgAADhCAYAAAA..."  
  ...  
}
```



```
{  
  ...  
  "profilePicture": "https://img.domain.com/4AF980..."  
  ...  
}
```

Large Data Handling



```
{  
  ...  
  "profilePicture": "iVBORw0KGgoUgAADhCAYAAAA..."  
  ...  
}
```



```
{  
  ...  
  "profilePictureUri": "https://img.domain.com/4F0..."  
  ...  
}
```

Broken Authentication

Common Mistake #3

Weak Signing Key

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ...
            IssuerSigningKey = "secret123", // ✗ weak
            ...
        };
    });
```

Core Token Checks Disabled

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,           // ✗ off
            ValidateAudience = false,        // ✗ off
            ValidateLifetime = false,        // ✗ off
            ValidateIssuerSigningKey = false // ✗ key not checked
            // (no ValidAlgorithms set)
        };
    });
```

Mixed Environment Configuration



Environment	SigningKey	Audience	Issuer
Development	NmQ5Njl2YWYtMDUyNi00MmL...	AppUser	AppName
Testing	NmQ5Njl2YWYtMDUyNi00MmL...	AppUser	AppName
Staging	NmQ5Njl2YWYtMDUyNi00MmL...	AppUser	AppName
Production	NmQ5Njl2YWYtMDUyNi00MmL...	AppUser	AppName

Mixed Environment Configuration



Environment	SigningKey	Audience	Issuer
Development	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser-dev	https://login-dev.appname.local
Testing	YWViYThiYzltZDI3Ny00MzUzLFk...	AppUser-test	https://login-test.appname.internal
Staging	ZTQyMmM1ZWUtNTNINy00ZjL...	AppUser-staging	https://login-staging.appname.com
Production	OTdiNDdmMjI5NGFINDAxNDMT...	AppUser	https://login.appname.com

Long-lived Access Tokens



Access tokens last hours or days

Refresh tokens never rotate

Long-lived Access Tokens



Short access token TTL (5–15 min)

Rotate refresh tokens and **revoke the old** on use

Invalidate all tokens on password reset

Insecure Cookies



```
.AddCookie(o =>
{
    o.Cookie.Name = ".app.auth";
    o.Cookie.HttpOnly = false; // X JS can read cookie
    o.Cookie.SecurePolicy = CookieSecurePolicy.None; // X sent over HTTP
    o.Cookie.SameSite = SameSiteMode.None; // X CSRF risk if not Secure
    o.ExpireTimeSpan = TimeSpan.FromDays(30); // X very long-lived
    o.SlidingExpiration = false; // X no refresh-on-use
});
```

Insecure Cookies



```
.AddCookie(o =>
{
    o.Cookie.Name = ".app.auth";
    o.Cookie.HttpOnly = true; // ✓ not readable by JS
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always; // ✓ HTTPS only
    o.Cookie.SameSite = SameSiteMode.Lax; // ✓ mitigates CSRF
    o.ExpireTimeSpan = TimeSpan.FromMinutes(30); // ✓ short-lived
    o.SlidingExpiration = true; // ✓ extend on active use
    o.Events = new CookieAuthenticationEvents
    {
        // Optional: add extra validation, e.g., check revoked sessions
    };
});
```

Misconfiguration and Insecure Defaults

Common Mistake #4

Security Headers

```
var builder = WebApplication.CreateBuilder(args); // default kestrel config
```

Security Headers



```
builder.WebHost.ConfigureKestrel(o => o.AddServerHeader = false);  
app.Use(async (ctx, next) =>  
{  
    ctx.Response.Headers["X-Content-Type-Options"] = "nosniff";  
    ctx.Response.Headers["Referrer-Policy"] = "no-referrer";  
    ctx.Response.Headers["Permissions-Policy"] = "geolocation=()";  
    await next();  
});
```

Proxy and Load Balancer Awareness



Behind a proxy but not forwarding scheme or host.

Proxy and Load Balancer Awareness



```
using Microsoft.AspNetCore.HttpOverrides;
app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedFor |
                       ForwardedHeaders.XForwardedProto |
                       ForwardedHeaders.XForwardedHost,
    KnownProxies = { System.Net.IPAddress.Parse("10.0.0.5") }
});
```


CORS That Is Not Wide Open



```
builder.Services.AddCors(p => p.AddPolicy("open", c => c
    .AllowAnyOrigin()
    .AllowAnyHeader()
    .AllowAnyMethod()
    .AllowCredentials()));
```

CORS That Is Not Wide Open



```
builder.Services.AddCors(p => p.AddPolicy("spa", c => c
    .WithOrigins("https://app.example.com")
    .WithHeaders("Content-Type", "Authorization")
    .WithMethods("GET", "POST", "PUT", "DELETE")
    .AllowCredentials()));
app.UseCors("spa");
```

OpenAPI in Production



```
app.UseSwagger();  
app.UseSwaggerUI();  
app.MapHealthChecks("/healthz"); // simple liveness
```

OpenAPI in Production



```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapSwagger().RequireAuthorization("Docs.Read");
app.MapHealthChecks("/healthz"); // simple liveness
```

Cookies



```
.AddCookie(o => {  
    o.Cookie.HttpOnly = false;  
    o.Cookie.SecurePolicy = CookieSecurePolicy.None;  
    o.Cookie.SameSite = SameSiteMode.None;  
});
```

Cookies



```
.AddCookie(o => {  
  o.Cookie.HttpOnly = true;  
  o.Cookie.SecurePolicy = CookieSecurePolicy.Always;  
  o.Cookie.SameSite = SameSiteMode.Lax; // Strict if same-site  
  o.SlidingExpiration = true;  
});
```

JSON Binding and Response Shaping



Bind entities directly

Unknown JSON fields ignored

JSON Binding and Response Shaping



```
services.AddControllers().AddJsonOptions(o =>  
    o.JsonSerializerOptions.UnmappedMemberHandling =  
        System.Text.Json.Serialization.JsonUnmappedMemberHandling.Disallow);
```


Unrestricted Resource Consumption

Common Mistake #5

Paging and Query Shape



```
// Unbounded read: client controls page size
```

```
[HttpGet] public Task<List<Order>> Get(int page = 1, int pageSize = 5000)  
    => db.Orders.Skip((page-1)*pageSize).Take(pageSize).ToListAsync();
```

Paging and Query Shape



```
const int MaxPageSize = 100; // server-enforced cap
[HttpGet] public async Task<IResult> Get(int page = 1, int pageSize = 50,
CancellationTokens ct = default)
{
    pageSize = Math.Clamp(pageSize, 1, MaxPageSize);
    var items = await db.Orders
        .AsNoTracking()
        .OrderByDescending(o => o.CreatedUtc)
        .Select(o => new OrderDto { Id = o.Id, Total = o.Total })
        .Skip((page-1)*pageSize).Take(pageSize)
        .ToListAsync(ct);
    return Results.Ok(items);
}
```

Rate Limiting



```
// No throttling; login and search endpoints can be hammered  
app.MapPost("/login", LoginHandler);  
app.MapGet("/search", SearchHandler);
```

Rate Limiting



```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.AddPolicy("tight", ctx => RateLimitPartition.GetTokenBucketLimiter(
        ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new TokenBucketRateLimiterOptions
        {
            TokenLimit = 5,                // max in bucket
            TokensPerPeriod = 5,           // refill each period
            ReplenishmentPeriod = TimeSpan.FromMinutes(1),
            AutoReplenishment = true
        }
    ));
});

...

// Apply the single policy to just the sensitive endpoint(s)
app.UseRateLimiter();
app.MapPost("/login", LoginHandler).RequireRateLimiting("tight");
app.MapGroup("/auth").RequireRateLimiting("tight"); // apply to many at once
```

Request Body Size



Kestrel Default is 30 MB

Request Body Size



```
builder.WebHost.ConfigureKestrel(o =>  
    o.Limits.MaxRequestBodySize = 2 * 1024 * 1024); // 2 MB
```

Verbose Errors & Logging Leaks

Common Mistake #6

Error Handling in Production



```
app.UseDeveloperExceptionPage();
```

Error Handling in Production



```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/error");

    app.Map("/error", () => Results.Problem(
        statusCode: 500,
        title: "An error occurred")); // keep generic
}
```

ProblemDetails, Not Stack Traces



```
//  Shows stack traces in every environment
```

```
app.UseDeveloperExceptionPage();
```

```
app.MapGet("/boom", () => throw new InvalidOperationException("DB timeout..."));
```

```
app.Run();
```

ProblemDetails, Not Stack Traces



```
app.MapGet("/boom", () => throw new InvalidOperationException("DB timeout..."));

// ✓ ProblemDetails endpoint (generic, no stack trace)
app.Map("/error", (HttpContext ctx) =>
{
    // Optional: attach a correlation id for support
    var cid = ctx.TraceIdentifier;
    return Results.Problem(
        statusCode: StatusCodes.Status500InternalServerError,
        title: "An error occurred",
        extensions: new Dictionary<string, object?> { ["correlationId"] = cid }
    );
});

app.Run();
```

Redact Secrets In Logs



```
builder.Services.AddHttpLogging(); // ✗ logs lots of headers/body by default
```

```
builder.Logging.AddConsole();
```

```
builder.Logging.SetMinimumLevel(LogLevel.Information);
```

```
optionsBuilder
```

```
    .EnableSensitiveDataLogging() // ✗ logs PII and secrets in SQL params
```

```
    .EnableDetailedErrors()
```

```
    .LogTo(Console.WriteLine, LogLevel.Information); // ✗ broad, unfiltered
```

Redact Secrets In Logs



```
// HTTP logging: safe fields only
b.Services.AddHttpLogging(o =>
    o.LoggingFields = HttpLoggingFields.RequestPath |
                    HttpLoggingFields.RequestMethod |
                    HttpLoggingFields.ResponseStatusCode);

// EF Core: no sensitive data logging; show SQL shape in Dev only
b.Services.AddDbContext<AppDbContext>(opts =>
{
    if (b.Environment.IsDevelopment())
        opts.EnableDetailedErrors()
            .LogTo(Console.WriteLine,
                new[] { DbLoggerCategory.Database.Command.Name },
                LogLevel.Information);
    // Prod: default (no LogTo, no SensitiveDataLogging)
});
```

Redact Secrets In Logs

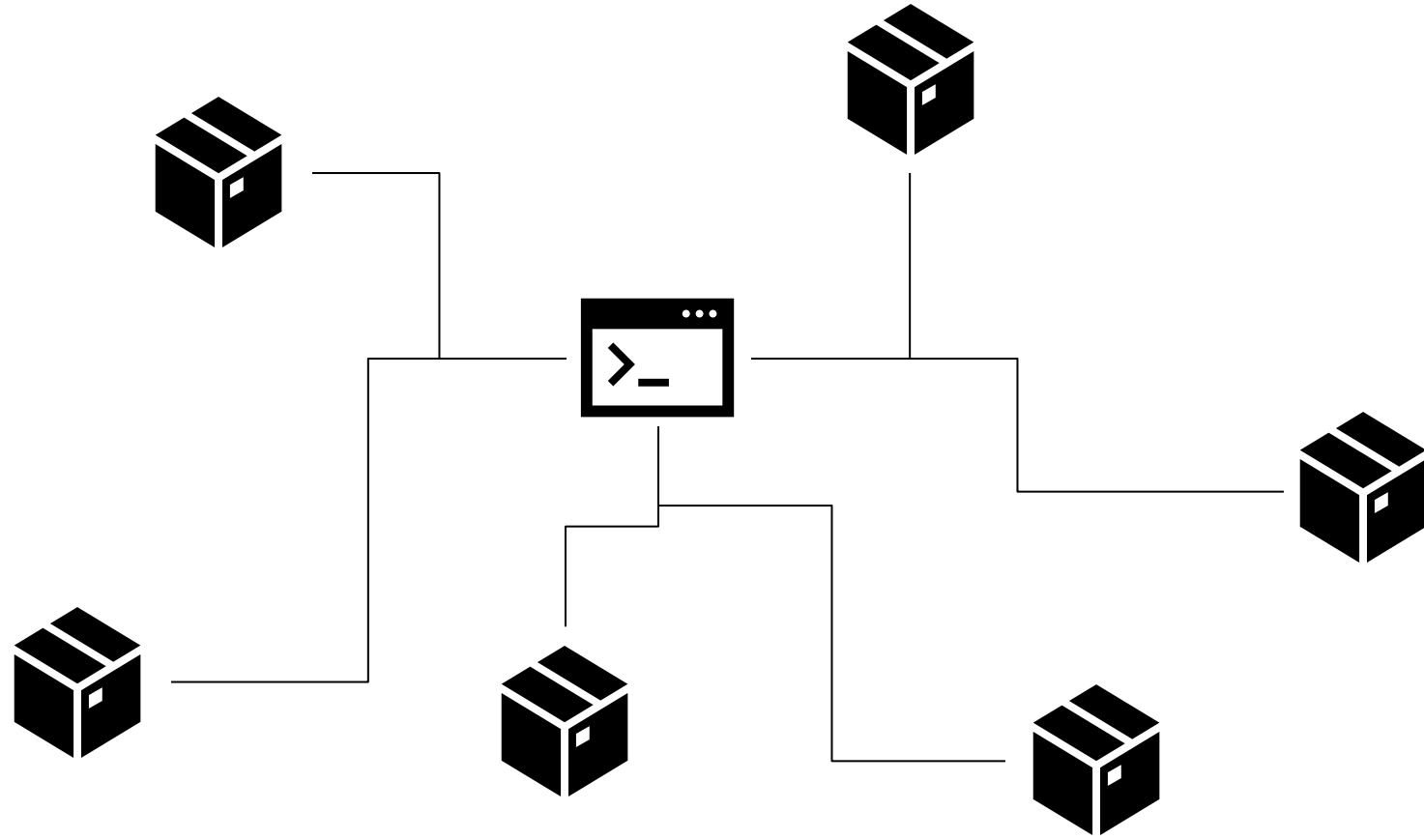


```
// Redact at your logger (example: Serilog)
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .Destructure.ByMaskingProperties("password",
        "authorization",
        "cookie",
        "set-cookie")
    .CreateLogger();
```

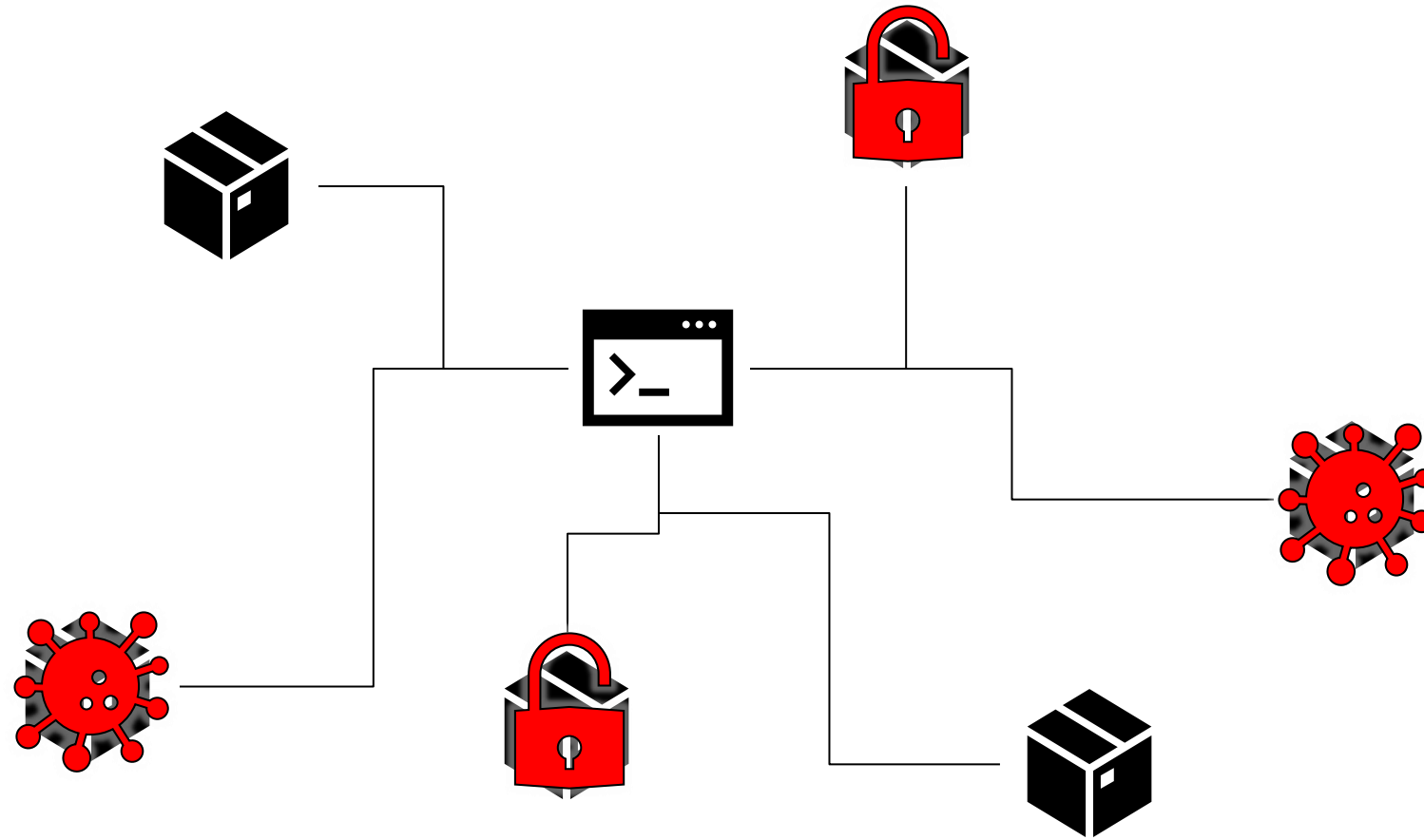
Dependency and Supply-Chain Risks

Common Mistake #7

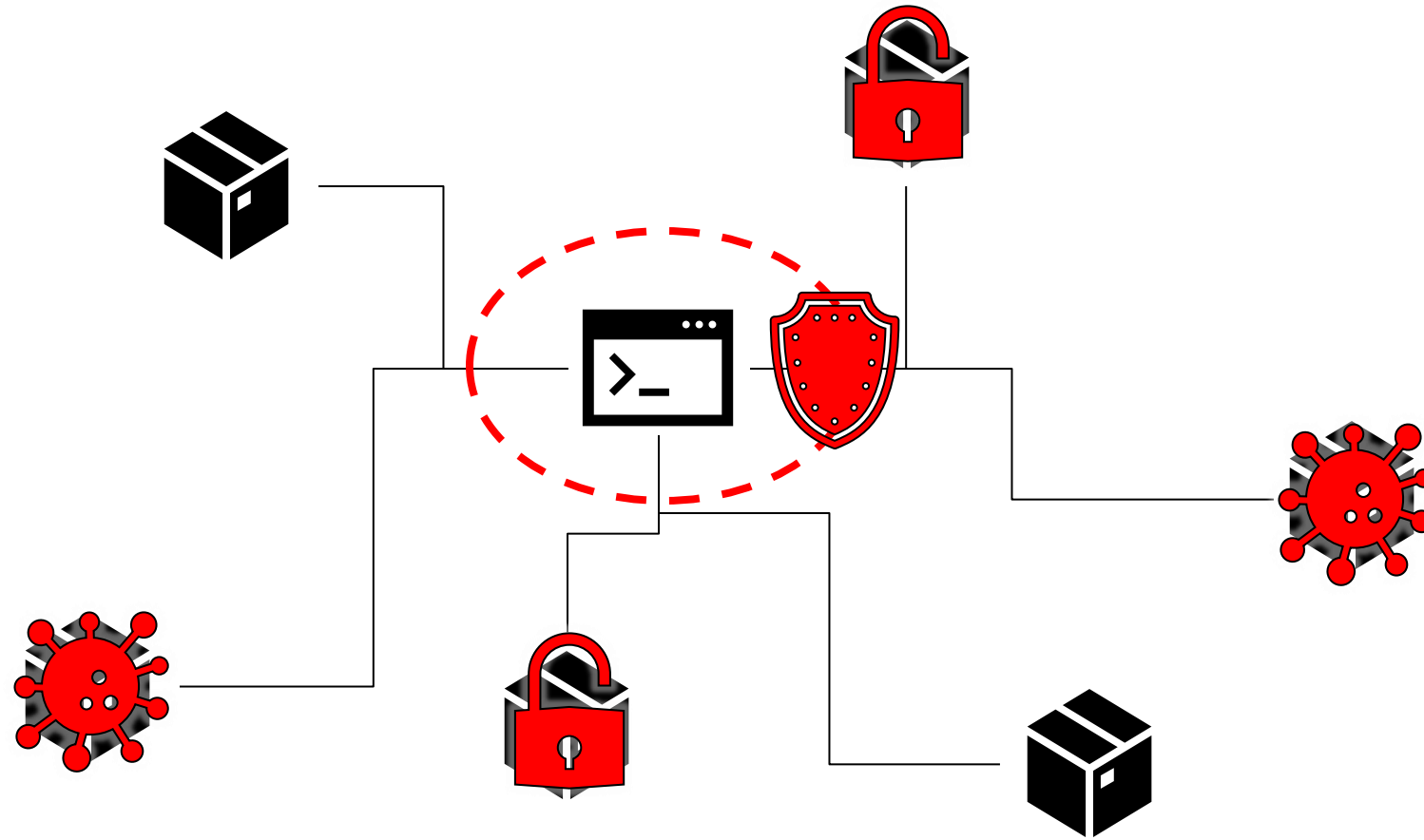
Security



Security



Security



Security

The screenshot shows the NuGet Package Manager interface. On the left, a list of packages is displayed, including Newtonsoft.Json, Newtonsoft.Json.Bson, Newtonsoft.Json.Schema, Microsoft.AspNetCore.Mvc.Newtonsoft.Json, Swashbuckle.AspNetCore.Newtonsoft, GraphQL.Newtonsoft.Json, RestSharp.Newtonsoft.Json, App.Metrics.Formatters.Json, NServiceBus.Newtonsoft.Json, Refit.Newtonsoft.Json, and RestSharp.Serializers.Newtonsoft.Json. On the right, the details for the selected package, Newtonsoft.Json, are shown. The version 12.0.3 is selected. A red circle highlights the 'Vulnerabilities detected (1)' section, which indicates that the package version has at least one vulnerability with high severity. The details section also includes information about the package, such as the author (James Newton-King), license (MIT), and dependencies.

NuGet Package Manager: Sample

Package source: nuget.org

Newtonsoft.Json

Version: 12.0.3 Install

Vulnerabilities detected (1)

This package version has at least one vulnerability with high severity. It may lead to specific problems in your project. Try updating the package version.

Details

Advisory (high severity): <https://github.com/advisories/GHSA-5crp-9r3c-p9vr>

Options

Description

Json.NET is a popular high-performance JSON framework for .NET

Version: 12.0.3

Author(s): James Newton-King

License: MIT

Date published: Friday, November 8, 2019 (11/8/2019)

Project URL: <https://www.newtonsoft.com/json>

Report Abuse: <https://www.nuget.org/packages/Newtonsoft.Json/12.0.3/ReportAbuse>

Tags: json

Dependencies

- .NETFramework,Version=v2.0
 - No dependencies
- .NETFramework,Version=v3.5
 - No dependencies
- .NETFramework,Version=v4.0
 - No dependencies
- .NETFramework,Version=v4.5
 - No dependencies
- .NETPortable,Version=v0.0,Profile=Profile259
 - No dependencies
- .NETPortable,Version=v0.0,Profile=Profile328
 - No dependencies
- .NETStandard,Version=v1.0
 - Microsoft.CSharp (>= 4.3.0)
 - NETStandard.Library (>= 1.6.1)
 - System.ComponentModel.TypeConverter (>= 4.3.0)

Security

`dotnet list package --vulnerable`

```
eShopCoreMVC — zsh — 117x30
jonathantower@MacBookPro eShopCoreMVC % dotnet list package --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

Project `eShopCoreMVC` has the following vulnerable packages
[net6.0]:
Top-level Package      Requested  Resolved  Severity  Advisory URL
> Yarp.ReverseProxy    2.0.0     2.0.0     High      https://github.com/advisories/GHSA-jrjw-qgr2-wfcg

jonathantower@MacBookPro eShopCoreMVC %
```

Security

`dotnet list package --vulnerable`

```
jonathantower@MacBookPro eShopCoreMVC % dotnet list package --vulnerable

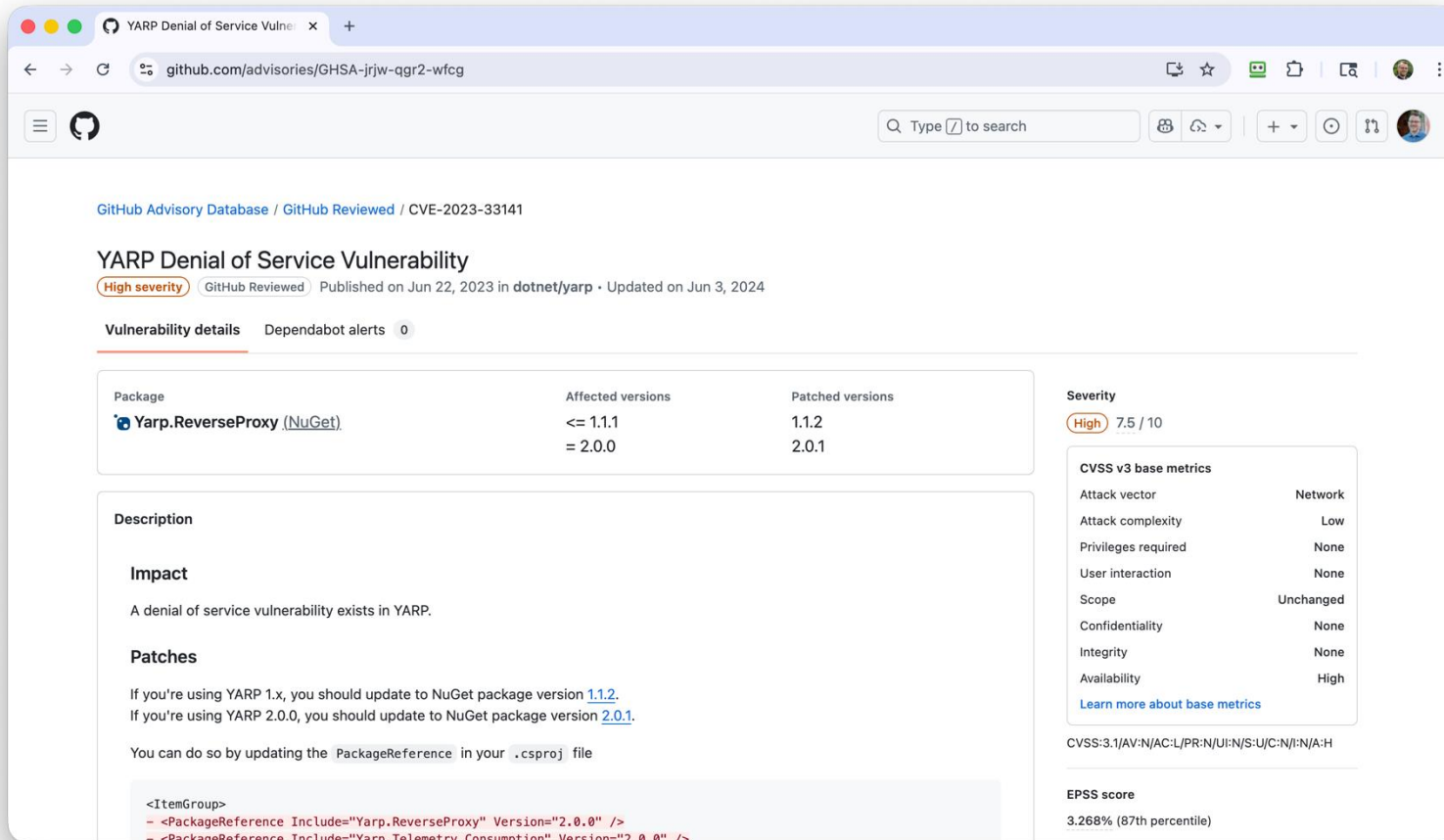
The following sources were used:
https://api.nuget.org/v3/index.json

Project `eShopCoreMVC` has the following vulnerable packages
[net6.0]:
Top-level Package      Requested  Resolved  Severity  Advisory URL
> Yarp.ReverseProxy     2.0.0     2.0.0     High      https://github.com/advisories/GHSA-jrjw-qgr2-wfcg

jonathantower@MacBookPro eShopCoreMVC %
```

Safety & Reliability: Security

dotnet list package --vulnerable



The screenshot shows a web browser displaying a GitHub Advisory Database entry for a Denial of Service (DoS) vulnerability in YARP. The page title is "YARP Denial of Service Vulnerability" with a "High severity" label. It was published on June 22, 2023, in the dotnet/yarp repository and updated on June 3, 2024. The vulnerability details table shows that the affected versions are <= 1.1.1 and = 2.0.0, while patched versions are 1.1.2 and 2.0.1. The severity is rated as High (7.5/10). The CVSS v3 base metrics are listed as: Attack vector (Network), Attack complexity (Low), Privileges required (None), User interaction (None), Scope (Unchanged), Confidentiality (None), Integrity (None), and Availability (High). The EPSS score is 3.268% (87th percentile). The description states that a denial of service vulnerability exists in YARP. The patches section provides instructions for updating NuGet package versions. A code snippet shows how to update the PackageReference in a .csproj file.

GitHub Advisory Database / GitHub Reviewed / CVE-2023-33141

YARP Denial of Service Vulnerability

High severity GitHub Reviewed Published on Jun 22, 2023 in dotnet/yarp • Updated on Jun 3, 2024

Vulnerability details Dependabot alerts 0

Package	Affected versions	Patched versions
 Yarp.ReverseProxy (NuGet)	<= 1.1.1 = 2.0.0	1.1.2 2.0.1

Description

Impact

A denial of service vulnerability exists in YARP.

Patches

If you're using YARP 1.x, you should update to NuGet package version [1.1.2](#).
If you're using YARP 2.0.0, you should update to NuGet package version [2.0.1](#).

You can do so by updating the `PackageReference` in your `.csproj` file

```
<ItemGroup>
  - <PackageReference Include="Yarp.ReverseProxy" Version="2.0.0" />
  - <PackageReference Include="Yarp.Telemetry.Consumption" Version="2.0.0" />
```

Severity

High 7.5 / 10

CVSS v3 base metrics

Metric	Value
Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

[Learn more about base metrics](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

EPSS score

3.268% (87th percentile)

Safety & Reliability: Security

dotnet list package --vulnerable

GitHub Advisory Database / GitHub Reviewed / CVE-2023-33141

YARP Denial of Service Vulnerability

High severity GitHub Reviewed Published on Jun 22, 2023 in dotnet/yarp • Updated on Jun 3, 2024

Vulnerability details Dependabot alerts 0

Package	Affected versions	Patched versions
Yarp.ReverseProxy (NuGet)	<= 1.1.1 = 2.0.0	1.1.2 2.0.1

Description

Impact

A denial of service vulnerability exists in YARP.

Patches

If you're using YARP 1.x, you should update to NuGet package version [1.1.2](#).
If you're using YARP 2.0.0, you should update to NuGet package version [2.0.1](#).

You can do so by updating the `PackageReference` in your `.csproj` file

```
<ItemGroup>
  <PackageReference Include="Yarp.ReverseProxy" Version="2.0.0" />
  <PackageReference Include="Yarp.Telemetry.Consumption" Version="2.0.0" />
</ItemGroup>
```

Severity

High 7.5 / 10

CVSS v3 base metrics

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

[Learn more about base metrics](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

EPSS score

3.268% (87th percentile)

Security

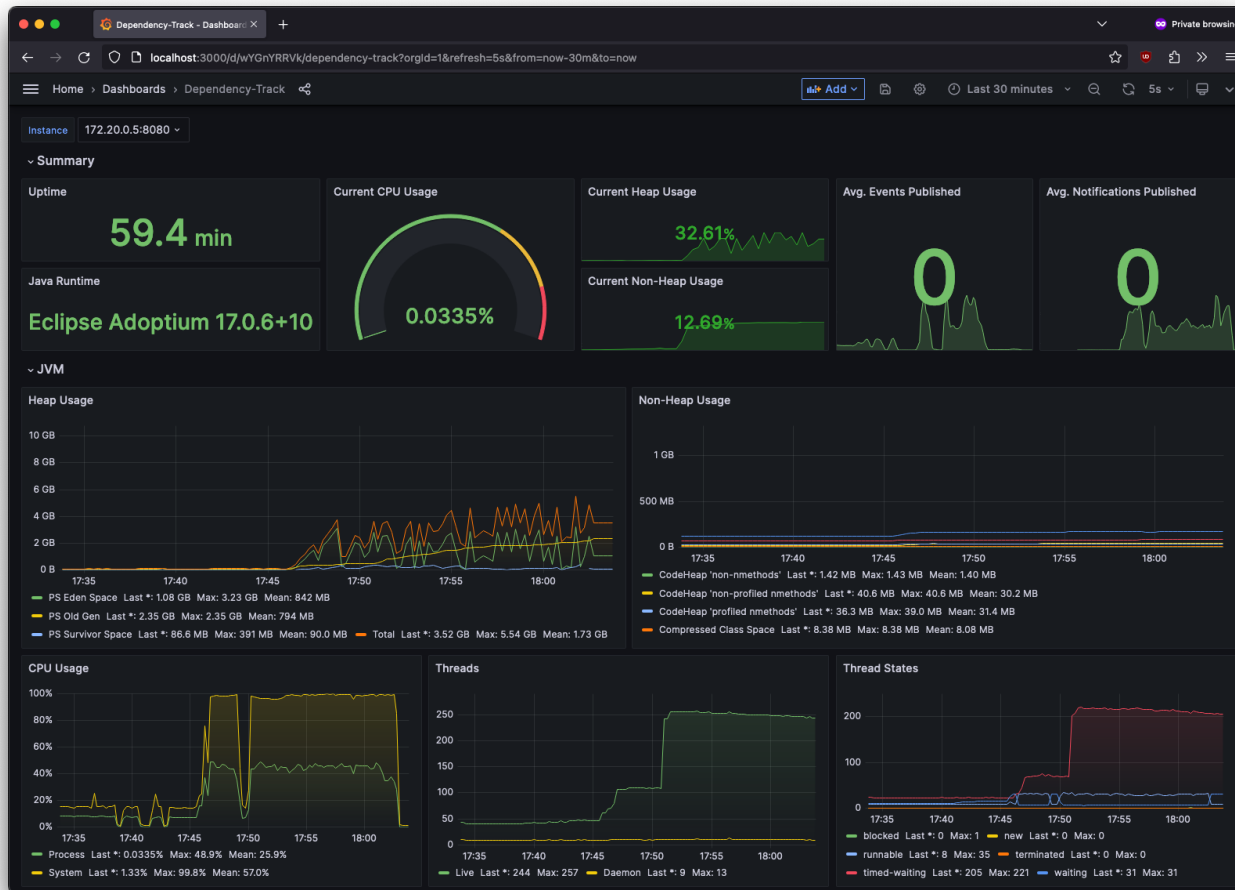
```
# npm audit report

ansi-html *
Severity: high
Uncontrolled Resource Consumption in ansi-html - https://github.com/advisories/GHSA-whgm-jr23-g3j9
fix available via `npm audit fix --force`
Will install @vue/cli-plugin-babel@3.12.1, which is a breaking change
node_modules/ansi-html
  webpack-dev-server 2.0.0-beta - 4.1.0
    Depends on vulnerable versions of ansi-html
    Depends on vulnerable versions of chokidar
    Depends on vulnerable versions of http-proxy-middleware
    Depends on vulnerable versions of yargs
  node_modules/webpack-dev-server
    @vue/cli-service <=5.0.0-beta.3
      Depends on vulnerable versions of @vue/cli-plugin-router
      Depends on vulnerable versions of @vue/cli-shared-utils
      Depends on vulnerable versions of copy-webpack-plugin
      Depends on vulnerable versions of cssnano
      Depends on vulnerable versions of globby
      Depends on vulnerable versions of webpack-dev-server
    node_modules/@vue/cli-service
      @vue/cli-plugin-babel 3.4.0 - 4.5.13
        Depends on vulnerable versions of @vue/cli-service
        Depends on vulnerable versions of @vue/cli-shared-utils
      node_modules/@vue/cli-plugin-babel
        @vue/cli-plugin-router <=4.5.13
          Depends on vulnerable versions of @vue/cli-service
        node_modules/@vue/cli-plugin-router
          @vue/cli-plugin-typescript <=5.0.0-alpha.1
            Depends on vulnerable versions of @vue/cli-service
            Depends on vulnerable versions of @vue/cli-shared-utils
            Depends on vulnerable versions of fork-ts-checker-webpack-plugin
            Depends on vulnerable versions of globby
          node_modules/@vue/cli-plugin-typescript
            @vue/cli-plugin-vuex <=4.5.13
              Depends on vulnerable versions of @vue/cli-service
            node_modules/@vue/cli-plugin-vuex
```

npm audit

Security

OWASP Dependency-Track



Codebase



SBOM Tool



SBOMs
(CycloneDX format)



OWASP Dependency-Track



TRAILHEAD
TECHNOLOGY PARTNERS

Transport Layer and Secrets Management

Common Mistake #8

HTTPS and TLS posture



```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(80)); // HTTP only  
app.UseHttpsRedirection();
```

HTTPS and TLS posture



```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(443, l => l.UseHttps()));  
app.UseHsts();
```

Secrets In Config



```
// appsettings.json
{
  "Jwt": {
    "SigningKey": "secret123" // X
  },
  "Db": {
    "Password": "P@ss!" // X
  }
}
```

Secrets In Config



// Dev: User Secrets

```
dotnet user-secrets set "Db:Password" "..."/> // not committed to repo
```

// Cloud: Azure Key Vault + Managed Identity

```
using Azure.Identity;
```

```
builder.Configuration
```

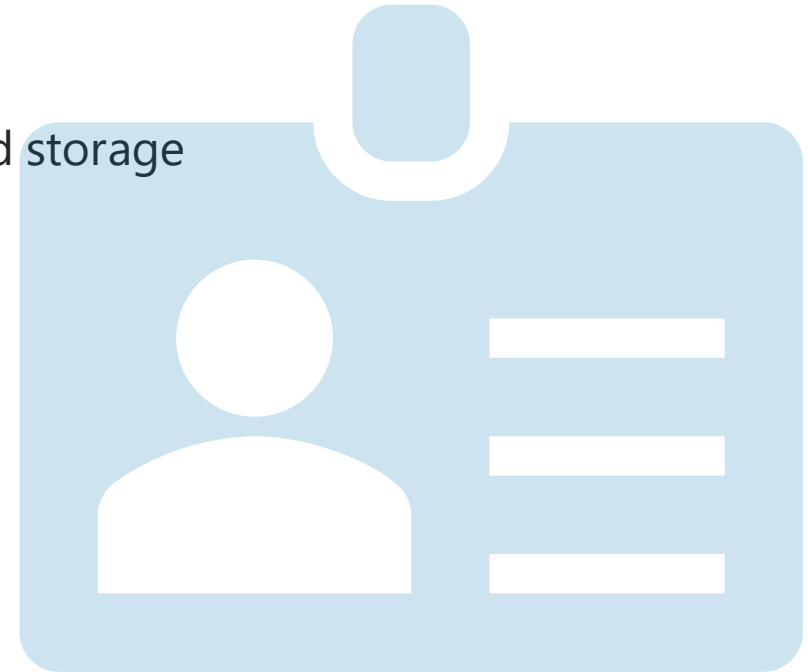
```
    .AddAzureKeyVault(new Uri("https://<vault>.vault.azure.net/"),
```

```
        new DefaultAzureCredential()); //  no static keys
```

Developer-Friendly Best Practices Checklist

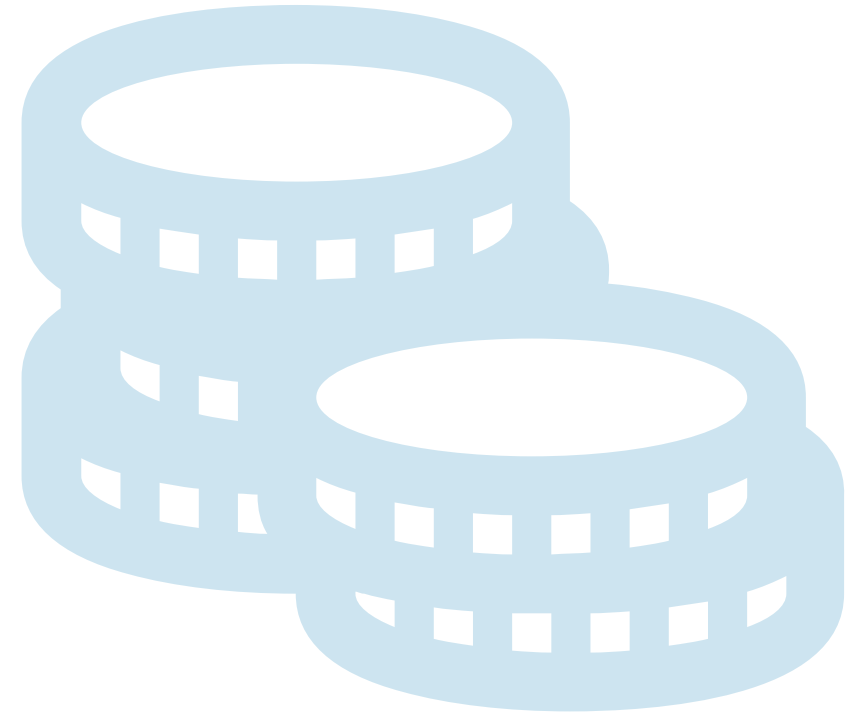
Authentication

- ✓ Use JWT
- ✓ Use proven implementations for auth, tokens, and password storage
- ✓ Use max retries & logout
- ✓ Encrypt sensitive data at rest



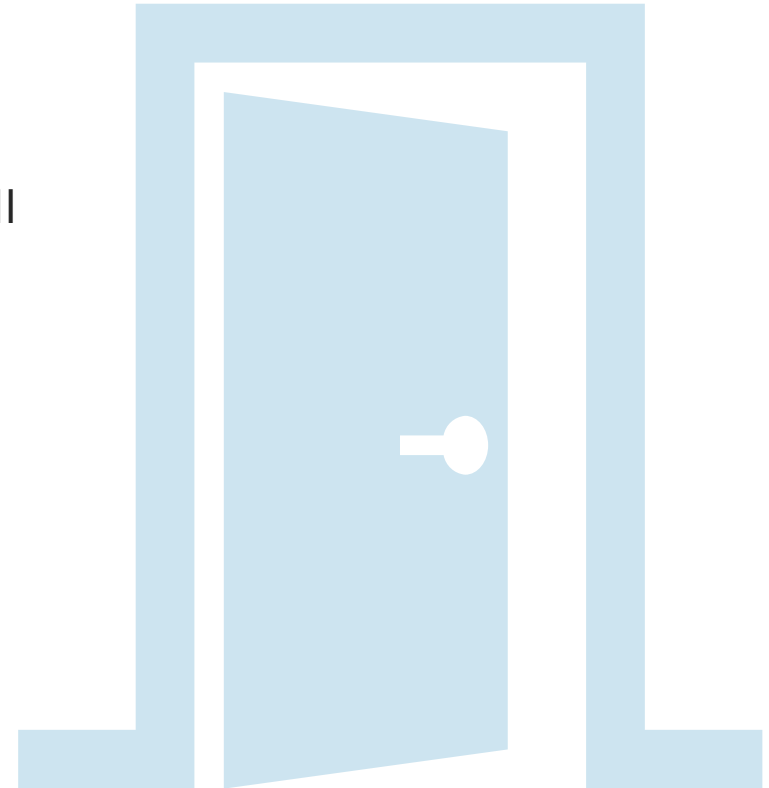
JSON Web Token

- ✓ Use a strong, random signing secret and rotate it
- ✓ Enforce the signing algorithm server-side
- ✓ Use short token lifetimes, refresh tokens
- ✓ Never store sensitive data in JWT payloads.
- ✓ Keep claims minimal



Access

- ✓ Rate-limit to prevent brute force attacks
- ✓ Enforce HTTPS (TLS 1.2+) with strong ciphers; verify Host/SNI
- ✓ Enable HSTS to prevent SSL stripping
- ✓ Disable directory indexing/listing
- ✓ Disable OpenAPI for private APIs
- ✓ Restrict private APIs to safelisted IPs/hosts



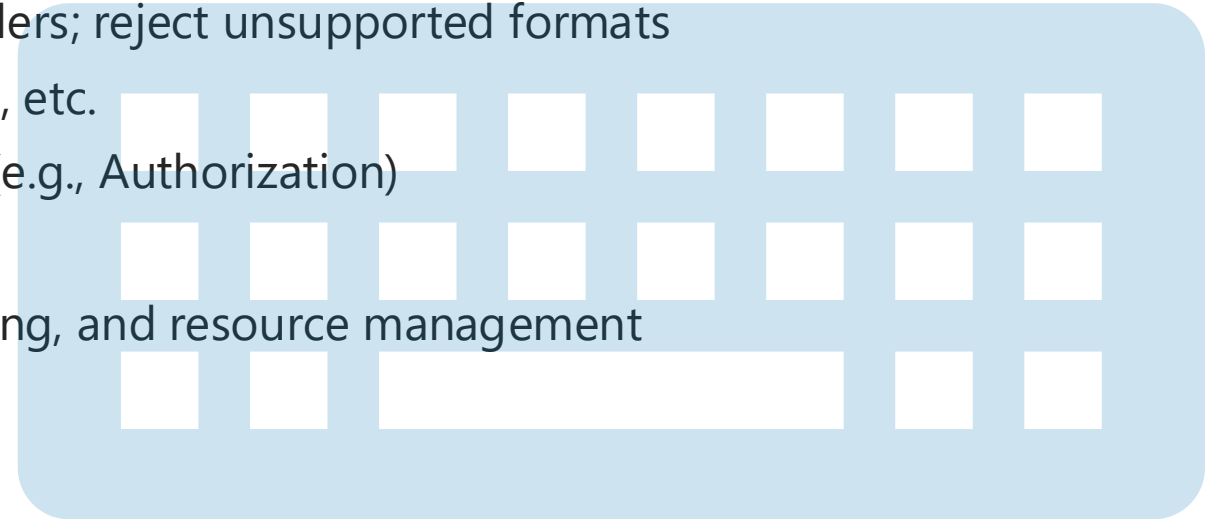
OAuth

- ✓ Validate redirect_uri server-side against a safelist
- ✓ Exchange authorization codes, not tokens (response_type=code)
- ✓ Use a random state parameter to prevent CSRF
- ✓ Define a default scope and validate requested scopes per application



Input

- ✓ Use correct HTTP methods (GET, POST, PUT/PATCH, DELETE); reject invalid ones
- ✓ Validate Accept and request Content-Type headers; reject unsupported formats
- ✓ Validate all user input to prevent XSS, SQLi, RCE, etc.
- ✓ Never pass sensitive data in URLs; use headers (e.g., Authorization)
- ✓ Encrypt data only on the server side
- ✓ Consider an API Gateway for caching, rate limiting, and resource management
 - Amazon API Gateway
 - Azure API Management
 - Cloudflare



Processing

- ✓ Protect all endpoints with authentication
- ✓ Use /me/... instead of exposing raw user IDs
- ✓ Use UUIDs instead of auto-increment IDs
- ✓ Disable XML entity parsing (prevent XXE)
- ✓ Disable entity expansion in XML/YAML (prevent "Billion Laughs" attacks)
- ✓ Use a CDN for file uploads
- ✓ Offload heavy processing to workers/queues for faster responses
- ✓ Turn off debug mode in production
- ✓ Enable non-executable stacks when supported



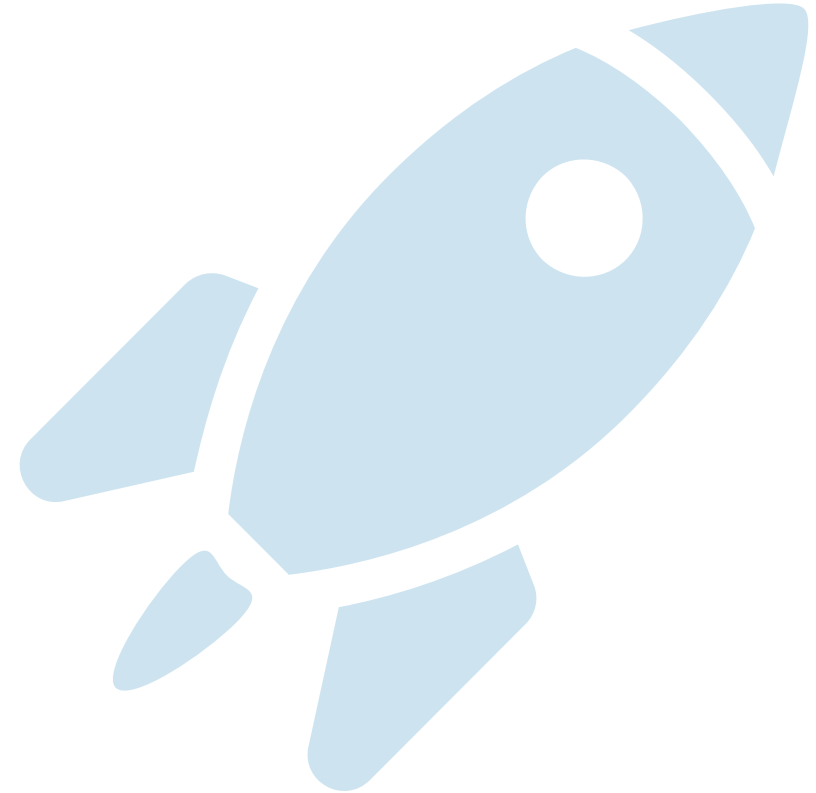
Output

- ✓ Send security headers (X-Content-Type-Options, X-Frame-Options, CSP)
- ✓ Remove fingerprinting headers (X-Powered-By, Server, etc.)
- ✓ Set the correct Content-Type in responses
- ✓ Never return sensitive data in responses
- ✓ Use proper HTTP status codes for each operation



CI & CD

- ✓ Cover design and code with unit/integration tests
- ✓ Require peer code reviews (no self-approval)
- ✓ Scan all components and dependencies before release
- ✓ Continuously run static/dynamic security tests
- ✓ Monitor dependencies/OS for known vulnerabilities
- ✓ Plan rollback strategies for deployments



Monitoring

- ✓ Centralize logins across all services
- ✓ Monitor traffic, errors, and requests with agents
- ✓ Set up alerts (SMS, Slack, Email, etc.)
- ✓ Avoid logging sensitive data (passwords, cards, PINs)
- ✓ Use IDS/IPS to monitor API requests and instances



The P.A.S.S. Framework

- P = Purpose-built
- A = Authenticate & Authorize
- S = Shape your data (API = UI)
- S = Secure defaults

Before You Ship...

- Ask yourself: does my API P.A.S.S. the test?

Summing Up

1. todo

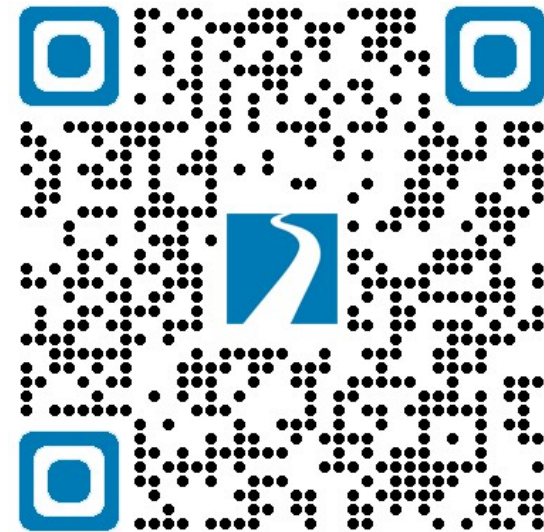


Thanks! Questions?

Jonathan "J." Tower

- Microsoft MVP in .NET
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- [jtowermi](#)
- Jonathan "J." Tower

EXPERT Consultation



bit.ly/th-offer

github.com/trailheadtechnology/api-security